

Lab Assignment 01



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Loops, String, Arrays, OOP Basics
Number of Tasks:	10 [Classwork: 5, Homework: 5]

Lab Policy: [Lab Policy Student Version - Summer 2025 Onward](#)
Must Submit the Lab Agreement Form: [Form Link](#)

CLASSWORK

Task 1

Write a Java program that takes 10 inputs from the user in a loop, and displays the sum, average, minimum and maximum of **Only the positive odd numbers** from those numbers. If no such numbers are found, then display the message “No odd positive numbers found”.

Sample Input	Sample Output
1 4 2 9 2 -4 3 -1 0 1	Sum = 14 Minimum = 1 Maximum = 9 Average = 3.5
34 -11 50 24 -24 2 -4 0 8 12	No odd positive numbers found
23 2 -4 0 8 12 34 -11 53 21	Sum = 97 Minimum = 21 Maximum = 53 Average = 32.333333333333336

Task 2

Write a Java program that takes TWO string inputs (containing exactly one word in each string) from the user. Concatenate those two strings with a single space in between them. Generate a number **which is the sum of all the letters in that concatenated string** where A = 65, Z = 90, a = 97, and z = 122. Your task is to print that concatenated string and the number generated from that string.

Sample Input	Output
Hello123 Wo%%rld	Hello123 Wo%%rld 1020
Ja12-va CHOWD+ HURY	Ja12-va CHOWD+ HURY 1087

Task 3

Write a Java program that asks the user the length of an array (N) then takes N number of doubles as elements for the array as input. First, remove the consecutive duplicate elements from the original array **to form a new array**. Then print the number of elements removed from the original array.

Sample Input	Sample Output
N = 8 Please enter the elements of the array: 5.2 2.7 1.0 1.0 2.7 3.5 3.5 3.5	New Array: 5.2 2.7 1.0 2.7 3.5 Removed elements : 3

Task 4

Design the “**Student**” class so that the main method prints the following:

Tester Class	Output
<pre>public class StudentTester{ public static void main(String [] args){ Student s1 = new Student(); System.out.println("Name of the Student: "+s1.name); System.out.println("ID of the Student: "+s1.id); s1.name = "Bob"; s1.id = 123; System.out.println("Name of the Student: "+s1.name); System.out.println("ID of the Student: "+s1.id); } }</pre>	<pre>Name of the Student: Default ID of the Student: 0 Name of the Student: Bob ID of the Student: 123</pre>

Task 5

Consider the following class:

```
public class Human{
    public int age;
    public double height;
}
```

Show the output of the following sequence of statements:

[illegible]

For this course, we'll be using **DrJava** as IDE for Java Coding:

[Link to JDK and DrJava](#)

Drjava Installation Guide:

<https://www.youtube.com/watch?v=Gss9sL3Q-8s>

HOMEWORK

Task 1

Write a java program that takes 2 integer numbers as input and calculates how many prime numbers exist between them.

Sample Input	Sample Output
10 15	There are 2 prime numbers between 10 and 15.
150 100	There are 10 prime numbers between 100 and 150.

Task 2

Write a Java program that takes a string input in small letters from the user and prints the previous alphabet in sequence for each alphabet found in the input.

Sample Input	Output
wxyz	vwxxy
thecow	sgdbnv
abcd	zabc

Task 3

Write a Java program that will take an integer number N from the user and create an integer array by taking N numbers from the user. Print how many times each number appears in the array.

Sample Input	Sample Output
N = 5 6	6 - 2 times 15 - 2 times

15 14 15 6	14 - 1 times
N = 6 -5 10 14 10 -7 10	-5 - 1 times 10 - 3 times 14 - 1 times -7 - 1 times

Task 4

Design the **CSECourse** class to generate the correct output from the driver code provided below:

Driver Code	Output
<pre> public class CourseTester{ public static void main(String args []){ CSECourse c1 = new CSECourse(); System.out.println("Course Name: "+c1.courseName); System.out.println("Course Code: "+c1.courseCode); System.out.println("Credit: "+c1.credit); } } </pre>	Course Name: Programming Language II Course Code: CSE111 Credit: 3

Task 5

Consider the following class:

```
public class Student{
    public String name;
    public double cgpa;
}
```

Show the output of the following sequence of statements:

[illegible]

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Write a Java program that will take an integer number N from the user and create an integer array by taking N numbers from the user. Then take another number from the user and create a new array by removing that number from the input array. Finally, print the new array.

Sample Input	Sample Output
N = 5 23 100 0 56 -34 Remove Element = 100	Input array: 23 100 0 56 -34 New array: 23 0 56 -34
N = 4 -5 10 2 -7 Remove Element = 43	Input array: -5 10 2 -7 Element not found

Task 2

Write a program that reads 5 numbers into an array and prints the smallest and largest number and their location in the array.

Sample Input	Sample Output
7 13 2 10 6	The largest number 13 was found at location 1. The smallest number 2 was found at location 2.

2 4 -5 12 3	The largest number 12 was found at location 3. The smallest number -5 was found at location 2.
-------------------------	---

Task 3

Write a program that asks the user how many numbers to take. Then, it takes that many numbers in an array and prints the median value.

[How to Find the Median Value: <http://www.mathsisfun.com/median.html>]

Sample Input	Sample Output
5 10 50 40 20 30	The median is 30. Explanation: 30 falls in middle 10, 20, 30, 40, 50
4 30 10 40 20	The median is 25. Explanation: $(20+30)/2=25$ (average of two middle values from 10, 20, 30, 40).

Task 4

Write a Java program that asks the user for the length of an array and then creates an integer array of that length by taking inputs from the user. Then, reverse the **original array without** creating any new array and print it. [**In-place reverse**]

Sample Input	Sample Output
Enter the length of the array: 5 7 -31	100 97 344 -31 7

344	
97	
100	

Task 5

Design the **Player** class with the necessary properties so that the given Driver code produces the expected output.

Driver Code	Output
<pre> public class PlayerTester{ public static void main(String args[]){ Player player1 = new Player(); player1.name = "Ronaldo"; player1.jersey_number = 9; player1.position = "Striker"; System.out.println("Name of the Player: "+ player1.name); System.out.println("Jersey Number of player: "+ player1.jersey_number); System.out.println("Position of player: "+ player1.position); System.out.println("====="); Player player2 = new Player(); player2.name = "Neuer"; player2.jersey_number = 1; player2.position = "Goal Keeper"; System.out.println("Name of the player: "+ player2.name); System.out.println("Jersey Number of player: "+ player2.jersey_number); System.out.println("Position of player: "+ player2.position); } } </pre>	<pre> Name of the Player: Ronaldo Jersey Number of player: 9 Position of player: Striker ===== Name of the player: Neuer Jersey Number of player: 1 Position of player: Goal Keeper </pre>

Lab Assignment 02



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	OOP Basics, Instance Variable and Instance Method
Number of Tasks:	12 (Classwork: 06, Homework: 06)

[Submit all the Coding Tasks (Homework: Task 1 to 4) in the Google Form shared on buX before the next lab. Submit the Tracing Tasks (Homework: Task 5 to 6) handwritten to your Lab Instructors at the beginning of the lab]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

You are given the following “**University**” class:

```
public class University{  
    public String name;  
    public String country;  
}
```

Now write a Java **tester** class named “**UniversityTester**”.

- a. Write the main method and create 2 objects of **University** class and print the location of the objects and print the instance variables of the objects. Are the location of the objects the same?
- b. Now change the instance variables of the first object.
 name = “Imperial College London”
 country = “England”

Now change the instance variables of the second object.

name = “BRAC University”
country = “Bangladesh”

Now check if the instance variables of both objects have changed or not and whether the instance variables of both objects are of the same value or not.

Task 2

Complete the “Cat” class so the main method produces the following output:

Test Class	Output
<pre>public class Test7{ public static void main(String [] args){ Cat c1 = new Cat(); System.out.println("1====="); c1.printCat(); c1.color = "Black"; System.out.println("2====="); c1.printCat(); c1.color = "Brown"; c1.action = "jumping"; System.out.println("3====="); c1.printCat(); } }</pre>	<pre>1===== White cat is sitting 2===== Black cat is sitting 3===== Brown cat is jumping</pre>

Task 3

Design the **Course** class to generate the correct output from the driver code provided below:

Driver Code	Output
<pre>public class Tester3{ public static void main(String[] args) { Course c1 = new Course(); Course c2 = new Course(); c1.updateDetails("Programming Language I","CSE110", 3); System.out.println("===== 1 ====="); c1.displayCourse(); c2.updateDetails("Data Structures","CSE220", 3); System.out.println("===== 2 ====="); c2.displayCourse(); c1.updateDetails("Programming Language II","CSE111", 3); System.out.println("===== 3 ====="); c1.displayCourse(); } }</pre>	<pre>===== 1 ===== Course Name: Programming Language I Course Code: CSE110 Course Credit: 3 ===== 2 ===== Course Name: Data Structures Course Code: CSE220 Course Credit: 3 ===== 3 ===== Course Name: Programming Language II Course Code: CSE111 Course Credit: 3</pre>

Task 4

Design the **CellPhone** class so that the **main** method of tester class can produce the following output:

Tester Class	Output
<pre>public class Tester4{ public static void main(String[]args){ CellPhone phone1 = new CellPhone(); phone1.printDetails(); phone1.model = "Nokia 1100"; System.out.println("1#####"); phone1.storeContact("Joy - 01834"); System.out.println("====="); phone1.printDetails(); System.out.println("2#####"); phone1.storeContact("Toya - 01334"); phone1.storeContact("Aayan - 01135"); System.out.println("====="); phone1.printDetails(); System.out.println("3#####"); phone1.storeContact("Sani - 01441"); System.out.println("====="); phone1.printDetails(); } }</pre>	<pre>Phone Model unknown Contacts Stored 0 1##### Contact Stored ===== Phone Model Nokia 1100 Contacts Stored 1 Stored Contacts: Joy - 01834 2##### Contact Stored Contact Stored ===== Phone Model Nokia 1100 Contacts Stored 3 Stored Contacts: Joy - 01834 Toya - 01334 Aayan - 01135 3##### Memory full. New contact can't be stored. ===== Phone Model Nokia 1100 Contacts Stored 3 Stored Contacts: Joy - 01834 Toya - 01334 Aayan - 01135</pre>

Task 5

1	<code>public class Task5 {</code>
2	<code> public int p = 3, y = 2, sum;</code>
3	<code> public void methodA(){</code>
4	<code> int x = 0, y = 0;</code>
5	<code> y = y + this.y;</code>
6	<code> x = sum + 2 + p;</code>
7	<code> sum = x + methodB(p, y) + y;</code>
8	<code> System.out.println(x + " " + y+ " " + sum);</code>
9	<code> }</code>
10	<code> public int methodB(int p, int n){</code>
11	<code> int x = 0;</code>
12	<code> y = y + (++p);</code>
13	<code> x = x + 2 + n;</code>
14	<code> sum = sum + x + y;</code>
15	<code> System.out.println(x + " " + y+ " " + sum);</code>
16	<code> return sum;</code>
17	<code> }</code>
18	<code>}</code>

Driver code:

<pre> public class Tester5 { public static void main(String [] args){ Task5 t1 = new Task5(); t1.methodA(); t1.methodA(); } } </pre>	Outputs		
	x	y	Sum

Task 6

1	public class Test6{
2	public int sum;
3	public int y;
4	public void methodA(){
5	int x=2, y =3;
6	int [] msg = {3, 7};
7	y = this.y + msg[0];
8	methodB(msg[1]++, msg[0]);
9	x = x + this.y + msg[1];
10	sum = x + y + msg[0];
11	System.out.println(x + " " + y+ " " + sum);
12	}
13	public void methodB(int mg2, int mg1){
14	int x = 0;
15	y = this.y + mg2;
16	x = x + 19 + mg1;
17	sum = this.sum + x + y;
18	mg2 = y + mg1;
19	mg1 = mg2 + x + 2;
20	System.out.println(x + " " + y+ " " + sum);
21	}
22	}

Driver code:

<pre> public class Tester6{ public static void main (String args[]){ Test6 t1 = new Test6(); t1.methodB(5,-8); Test6 t2 = new Test6(); t2.methodA(); } } </pre>	Outputs		

HOMEWORK

Task 1

Design the “**ImaginaryNumber**” class to generate the **output** given below:

Tester Class	Output
<pre>public class Tester7{ public static void main(String [] args){ ImaginaryNumber num1 = new ImaginaryNumber(); String p = num1.printNumber(); System.out.println(p); System.out.println("1*****"); num1.realPart=3; num1.imaginaryPart=7; System.out.println(num1.printNumber()); System.out.println("2*****"); ImaginaryNumber num2 = new ImaginaryNumber(); num2.realPart=1; num2.imaginaryPart=9; System.out.println(num2.printNumber()); } }</pre>	<pre>0 + 0i 1***** 3 + 7i 2***** 1 + 9i</pre>

Task 2

Implement the “**Assignment**” class with necessary properties, so that the given output is produced for the provided driver code.

Driver Class	Output
<pre>public class AssignmentTester{ public static void main(String [] args){ Assignment as1 = new Assignment(); as1.printDetails(); System.out.println("1-----"); as1.tasks = 11; as1.difficulty = "Moderate"; as1.submission = true; as1.printDetails(); System.out.println("2-----"); } }</pre>	<pre>Number of tasks: 0 Difficulty level: null Submission required: false 1----- Number of tasks: 11 Difficulty level: Moderate Submission required: true 2----- Assignment will not require submission</pre>

<pre> System.out.println(as1.makeOptional()); System.out.println("3-----"); as1.printDetails(); System.out.println("4-----"); Assignment as2 = new Assignment(); as2.tasks = 12; as2.difficulty = "Hard"; as2.submission = false; as2.printDetails(); System.out.println("5-----"); System.out.println(as2.makeOptional()); } } </pre>	<pre> 3----- Number of tasks: 11 Difficulty level: Moderate Submission required: false 4----- Number of tasks: 12 Difficulty level: Hard Submission required: false 5----- Submission is already not required </pre>
--	--

Task 3

Create an **Employee** class to provide the expected output.

- An employee will have a name, salary and designation.
- The name will be assigned inside the newEmployee() method
- Whenever a New Employee joins his/her salary will be **Tk. 30,000** and the designation will be **junior**.
- Employees with salaries greater than **Tk. 50,000** and **Tk. 30,000** need to pay **30%** and **10%** of salary as tax respectively.
- Employees can be promoted to **senior**, **lead** and **manager** positions. Based on their promotion they will get an increment of **Tk. 25,000**, **Tk. 50,000** and **Tk. 75,000** respectively.

Driver Code	Expected Output
<pre> public class Tester9{ public static void main(String[] args){ Employee emp1 = new Employee(); Employee emp2 = new Employee(); Employee emp3 = new Employee(); emp1.newEmployee("Harry Potter"); emp2.newEmployee("Hermione Granger"); emp3.newEmployee("Ron Weasley"); System.out.println("1 ====="); emp1.displayInfo(); System.out.println("2 ====="); emp2.displayInfo(); System.out.println("3 ====="); emp3.displayInfo(); System.out.println("4 ====="); emp1.calculateTax(); System.out.println("5 ====="); emp1.promoteEmployee("lead"); System.out.println("6 ====="); emp1.calculateTax(); System.out.println("7 ====="); emp1.displayInfo(); System.out.println("8 ====="); emp3.promoteEmployee("manager"); System.out.println("9 ====="); emp3.calculateTax(); System.out.println("10 ====="); emp3.displayInfo(); } } </pre>	<pre> 1 ===== Employee Name: Harry Potter Employee Salary: 30000.0 Tk Employee Designation: junior 2 ===== Employee Name: Hermione Granger Employee Salary: 30000.0 Tk Employee Designation: junior 3 ===== Employee Name: Ron Weasley Employee Salary: 30000.0 Tk Employee Designation: junior 4 ===== No need to pay tax 5 ===== Harry Potter has been promoted to lead New Salary: 80000.00 Tk 6 ===== Harry Potter Tax Amount: 24000.0 Tk 7 ===== Employee Name: Harry Potter Employee Salary: 80000.0 Tk Employee Designation: lead 8 ===== Ron Weasley has been promoted to manager New Salary: 105000.00 Tk 9 ===== Ron Weasley Tax Amount: 31500.0 Tk 10 ===== Employee Name: Ron Weasley Employee Salary: 105000.0 Tk Employee Designation: manager </pre>

Task 4

Implement the “**MobilePhone**” class with necessary properties to produce the given output for the provided driver code.

Driver Class	Output
<pre>public class MobilePhoneTester{ public static void main(String args []){ MobilePhone m1 = new MobilePhone(); MobilePhone m2 = new MobilePhone(); m1.setContactCapacity(5); m2.setContactCapacity(100); m1.details(); System.out.println("1-----"); m1.addContact("John", 9866); m1.addContact("Maria", 7865); System.out.println("2-----"); m1.details(); System.out.println("3-----"); m1.makeCall(9866); System.out.println("4-----"); m1.addContact("Henry", 2365); System.out.println("5-----"); m1.makeCall(7552); m1.makeCall(2365); System.out.println("6-----"); m1.addContact("Gomes", 4589); m1.addContact("Antony", 8421); m1.addContact("Tony", 5789); System.out.println("7-----"); m1.details(); } }</pre>	<pre>Total Contacts: 0 Contact List: 1----- The contact of John is added. The contact of Maria is added. 2----- Total Contacts: 2 Contact List: John:9866 Maria:7865 3----- Calling John . . . 4----- The contact of Henry is added. 5----- Calling 7552 . . . Calling Henry . . . 6----- The contact of Gomes is added. The contact of Antony is added. Storage Full!! 7----- Total Contacts: 5 Contact List: John:9866 Maria:7865 Henry:2365 Gomes:4589 Antony:8421</pre>

Task 5

1	public class B {
2	public int temp = 4;
3	public int sum, y, x;
4	public void methodA(int m){
5	int [] n = {2,5};
6	int x = 0;
7	y = m + this.methodB(x++,m)+(temp++);
8	x = this.x + 2 + n[0];
9	sum = sum + x + y;
10	n[0] = sum + 2;
11	System.out.println(n[0]+" " + x+ " " + sum);
12	}
13	public int methodB(int m, int n){
14	int y = 4 + this.y + m;
15	x = this.y + y + (++temp) - n;
16	sum = x + y + this.sum;
17	System.out.println(y+ " " + this.x + " " +sum);
18	return x;
19	}
20	}

<pre> public class Tester11 { public static void main(String [] args){ B t1 = new B(); t1.methodA(5); B t2 = new B(); t2.methodB(12, 2); } } </pre>	Outputs		

Task 6

1	public class A{
2	public int x = 3, y = 5, sum = 9;
3	public int methodA(int temp, int x){
4	this.x += (x++) + temp;
5	if(temp % 5 == 2){
6	sum += (this.y++) + temp;
7	}
8	else{
9	sum += 3;
10	if(y > 5) ++y;
11	}
12	System.out.println(this.x + " " + y + " " + sum);
13	return this.x;
14	}
15	public void methodB(int y){
16	int temp = (y++) + this.y;
17	this.y = (++temp) + methodA(temp, y) + x;
18	sum = y + (++this.x) + (temp++);
19	System.out.println(x + " " + y + " " + (sum++));
20	}
21	public void methodC(int y){
22	y = (this.x++) + sum + 3;
23	System.out.println(x + " " + y + " " + sum);
24	}
25	}

Driver code:

<pre> public class Test12{ public static void main(String [] args) { A a1 = new A(); a1.methodA(2, 4); a1.methodB(3); A a2 = new A(); a2.methodC(7); a1.methodB(3); } } </pre>	Outputs		

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Complete the **Bird** class so that main method produces the following **output**:

Test class	Output
<pre>public class BirdTest{ public static void main(String args[]) { Bird b1 = new Bird(); b1.name = "Parrot"; b1.flyUp(3); b1.makeNoise(); b1.flyDown(5); b1.flyDown(2); b1.flyDown(1); Bird b2 = new Bird(); b2.name = "Eagle"; b2.flyUp(5); b2.flyDown(5); b2.makeNoise(); } }</pre>	<pre>Parrot has flown up 3 feet. Squawk Parrot cannot fly down 5 feet. Parrot has flown down 2 feet. Parrot has flown down 1 feet and landed. Eagle has flown up 5 feet. Eagle has flown down 5 feet and landed. Squee</pre>

Task 2

Implement the “**ChickenBurger**” class with necessary properties, so that the given output is produced for the provided driver code.

[Note:

1. There are four available spice levels: **Mild**, **Spicy**, **Naga** and **Extreme**. You can store these values in a String array.
2. You might need to use the `.equals()` method to compare two string values.]

Driver Class	Output
<pre>public class BurgerMaker{ public static void main(String [] args){ ChickenBurger b1 = new ChickenBurger(); System.out.println(b1.bun); } }</pre>	<pre>Sesame 200 Less Not Set -----1----- Cannot serve now. Customize Spice</pre>

```

System.out.println(b1.price);
System.out.println(b1.sauceOption);
System.out.println(b1.spiceLevel);
System.out.println("-----1-----");
System.out.println(b1.serveBurger());
System.out.println("-----2-----");
b1.customizeSpiceLevel("Extreme Jhaal");
b1.customizeSpiceLevel("Spicy");
System.out.println("-----3-----");
System.out.println(b1.serveBurger());
System.out.println("-----4-----");
ChickenBurger b2 = new ChickenBurger();
b2.bun = "Brioche";
b2.price += 50;
b2.sauceOption = "Regular";
b2.customizeSpiceLevel("Naga");
System.out.println("-----5-----");
System.out.println(b2.serveBurger());
}
}

```

```

Level first.
-----2-----
This spice level is unavailable.
Spice level set to Spicy.
-----3-----
The burger is being served:-
Bun Type: Sesame
Price: 200
Sauce Option: Less
Spice Level: Spicy
-----4-----
Spice level set to Naga.
-----5-----
The burger is being served:-
Bun Type: Brioche
Price: 250
Sauce Option: Regular
Spice Level: Naga

```

Task 3

Design the **MagicPotion** class so that the following Tester class generates the expected output

- Potion cannot be activated if potion strength below 5

Driver Code	Outputs
<pre>public class MagicTester { public static void main(String[] args) { MagicPotion potion = new MagicPotion(); potion.setDetails("blue", 4); System.out.println("====="); potion.activate(); potion.describe(); System.out.println("====="); potion.boost(); potion.describe(); System.out.println("====="); potion.activate(); potion.describe(); System.out.println("====="); System.out.println("Is potion strong? " + potion.isStrong()); System.out.println("====="); potion.boost(); potion.activate(); potion.describe(); System.out.println("====="); System.out.println("Is potion strong? " + potion.isStrong()); System.out.println("====="); potion.boost(); System.out.println("Is potion strong? " + potion.isStrong()); System.out.println("====="); potion.weaken(); potion.weaken(); potion.weaken(); potion.describe(); } }</pre>	<pre>===== Potion colour: blue Potion strength: 4 Is active: false ===== Potion colour: blue Potion strength: 5 Is active: false ===== Potion colour: blue Potion strength: 5 Is active: true ===== Is potion strong? false ===== Already activated Potion colour: blue Potion strength: 6 Is active: true ===== Is potion strong? false ===== Is potion strong? true ===== Potion colour: blue Potion strength: 4 Is active: false</pre>

Task 4

1	public class TraceA{
2	public int x =0, y = 8, sum = 0;
3	public void methodA(int y, int x){
4	int [] arr = {4, 7};
5	this.x += x;
6	arr[0] = y++;
7	arr[1] += arr[1] % arr[0] * x;
8	if (y % 2 == 0){
9	sum = x + methodB(arr[0]++, arr[1], y) + this.x;
10	}
11	else{
12	sum = this.y + methodB(++arr[0], arr[1], x) + this.y;
13	}
14	System.out.println(x+ " " + arr[0] + " " + sum);
15	}
16	public int methodB(int a, int b, int x){
17	sum = sum % x;
18	if(a % b == x){
19	return this.y--;
20	}
21	this.x = a * b / x;
22	y += this.x % this.y;
23	System.out.println(a + " " + this.x + " " + y);
24	return y;
25	}
26	}

Driver code:

<pre> public class TesterX{ public static void main(String [] args){ TraceA t1 = new TraceA(); t1.methodA(4, 7); int x = t1.methodB(3,2,1); System.out.println(t1.y + " " + t1.sum + " " + x); } } </pre>	Output		

Lab Assignment 03



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Method Overloading, Constructor & Overloading
Number of Tasks:	12 (Classwork: 06, Homework: 06)

[Submit all the Coding Tasks (Homework: Task 1 to 4) in the Google Form shared on buX before the next lab. Submit the Tracing Tasks (Homework: Task 5 to 6) handwritten to your Lab Instructors at the beginning of the lab]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

Design the **Shape** class with necessary properties to produce the given output for the provided driver code.

Driver Class	Output
<pre>public class ShapeTester{ public static void main(String args []){ Shape circle = new Shape(); Shape triangle = new Shape(); Shape rectangle = new Shape(); circle.setParameters("Circle", 5); triangle.setParameters("Triangle", 4, 7); rectangle.setParameters("Rectangle", 2.4, 4.4); System.out.println(circle.details()); System.out.println("1-----"); System.out.println(triangle.details()); System.out.println("2-----"); System.out.println(rectangle.details()); } }</pre>	<pre>Shape Name: Circle Area: 78.54 1----- Shape Name: Triangle Area: 14.0 2----- Shape Name: Rectangle Area: 10.56</pre>

Task 2

Design the **Student** class in such a way that it produces the following output.

Driver Code	Expected Output
<pre>public class StudentTester{ public static void main(String[] args){ Student s1 = new Student("Harry", "CSE"); System.out.println(s1.name); s1.updateName("Harry Potter"); System.out.println(s1.name); System.out.println(s1.prog); s1.updateProgram("CS"); String var = s1.accessProgram(); System.out.println(var); } }</pre>	<pre>Harry Harry Potter CSE CS</pre>

Task 3

Design the **Shape2D** class in such a way that it produces the following output.

Driver Code	Expected Output
<pre>public class Shape2DTester { public static void main(String[] args) { Shape2D sq = new Shape2D(); System.out.println("-----1-----"); sq.area(); System.out.println("-----2-----"); Shape2D rectangle = new Shape2D(5, 6); System.out.println("-----3-----"); rectangle.area(); System.out.println("-----4-----"); Shape2D tri1 = new Shape2D(5, 6, "Triangle"); System.out.println("-----5-----"); tri1.area(); System.out.println("-----6-----"); Shape2D tri2 = new Shape2D(5, 6, 7); System.out.println("-----7-----"); tri2.area(); } }</pre>	<pre>A Square has been created with length: 5 -----1----- The area of the Square is: 25.00 -----2----- A Rectangle has been created with length: 5 and breath: 6 -----3----- The area of the Rectangle is: 30.00 -----4----- A Triangle has been created with height: 5 and base: 6 -----5----- The area of the Triangle is: 15.00 -----6----- A Triangle has been created with the following sides: 5, 6, 7 -----7----- The area of the Triangle is: 14.70</pre>

Task 4

Design the **Library** class with the necessary properties so that the given output is produced for the provided driver code.

Driver Code	Output
<pre>public class Tester5{ public static void main(String[] args) { Library a1 = new Library(3); System.out.println("1-----"); a1.addBook("Ice"); System.out.println("2-----"); a1.printDetail(); System.out.println("3-----"); a1.addBook("Emma"); a1.addBook("Wings"); a1.addBook("Next"); System.out.println("4-----"); a1.printDetail(); System.out.println("5-----"); Library a2 = new Library(4); System.out.println("6-----"); a2.addBook("Onnobhubon"); a2.addBook("Ami"); System.out.println("7-----"); a2.printDetail(); System.out.println("8-----"); a2.addBook("Deyal"); a2.addBook("Himu"); a2.addBook("Megher Upor Bari"); System.out.println("9-----"); a2.printDetail(); } }</pre>	<pre>A library has been created with capacity 3 1----- Book 'Ice' added to the library 2----- Maximum Capacity: 3 Total Books: 1 Book list: Ice 3----- Book 'Emma' added to the library Book 'Wings' added to the library Exceeds maximum capacity. You can't add more than 3 books 4----- Maximum Capacity: 3 Total Books: 3 Book list: Ice Emma Wings 5----- A library has been created with capacity 4 6----- Book 'Onnobhubon' added to the library Book 'Ami' added to the library 7----- Maximum Capacity: 4 Total Books: 2 Book list: Onnobhubon Ami 8----- Book 'Deyal' added to the library Book 'Himu' added to the library Exceeds maximum capacity. You can't add more than 4 books 9----- Maximum Capacity: 4 Total Books: 4 Book list: Onnobhubon Ami Deyal Himu</pre>

Task 5

Design the **TaxiLagbe** class with necessary properties to produce the given output for the provided driver code.

Driver Code	Output
<pre>public class TaxiTester{ public static void main(String[] args) { TaxiLagbe taxi1 = new TaxiLagbe("1010-01","Dhaka"); System.out.println("1-----"); taxi1.printDetails(); System.out.println("2-----"); taxi1.addPassenger("Wilson", 105); System.out.println("3-----"); taxi1.printDetails(); System.out.println("4-----"); taxi1.addPassenger("Walker", 100, "Wood", 200); System.out.println("5-----"); taxi1.printDetails(); System.out.println("6-----"); taxi1.addPassenger("Karen", 200); taxi1.addPassenger("Donald", 130); System.out.println("7-----"); taxi1.printDetails(); System.out.println("8-----"); TaxiLagbe taxi2 = new TaxiLagbe("1010-02"); System.out.println("9-----"); taxi2.addPassenger("Don", 115, "Parker", 215); System.out.println("10-----"); taxi2.printDetails(); } }</pre>	<pre>A new taxi has been created 1----- Taxi number: 1010-01 This taxi can cover Dhaka area Total Passenger: 0 Passenger List: Total collected fare: 0 Taka 2----- Dear Wilson! Welcome to TaxiLagbe 3----- Taxi number: 1010-01 This taxi can cover Dhaka area Total Passenger: 1 Passenger List: Wilson Total collected fare: 105 Taka 4----- Dear Walker! Welcome to TaxiLagbe Dear Wood! Welcome to TaxiLagbe 5----- Taxi number: 1010-01 This taxi can cover Dhaka area Total Passenger: 3 Passenger List: Wilson Walker Wood Total collected fare: 405 Taka 6----- Dear Karen! Welcome to TaxiLagbe Taxi Full! No more passengers can be added 7----- Taxi number: 1010-01 This taxi can cover Dhaka area Total Passenger: 4 Passenger List: Wilson Walker Wood Karen Total collected fare: 605 Taka 8----- A new taxi has been created 9----- Dear Don! Welcome to TaxiLagbe Dear Parker! Welcome to TaxiLagbe 10----- Taxi number: 1010-02 This taxi can cover Khulna area Total Passenger: 2 Passenger List: Don Parker Total collected fare: 330 Taka</pre>

Task 6

1	public class A{
2	public int temp = 3, sum = 9, y = 4, x = 0;
3	public A(){
4	int sum = 7;
5	y = temp - 5;
6	sum = temp + 2;
7	temp -= 2;
8	this.x = sum + temp + y;
9	}
10	public A(int y, int temp){
11	y = temp - 1 + x;
12	sum = temp + 2 -x;
13	temp-=2;
14	}
15	public void methodB(int m, int [] n){
16	int x = 0;
17	y = y + m + this.methodB(x,m) ;
18	x = this.x + 2 + (++n[0]) ;
19	sum = sum + x + y;
20	n[0] = sum + 2;
21	System.out.println(n[0] + " " + y+ " " + sum) ;
22	}
23	public int methodB(int m, int n){
24	int [] y = {10};
25	this.y = y[0] + this.y + m;
26	x = this.y + 2 + temp - n;
27	sum = x + y[0] + this.sum;
28	System.out.println(y[0]+ " "+ temp + " " + sum) ;
29	return y[0] ;
30	}
31	}

Driver Code	Output		
<pre> public class Tester6 { public static void main(String args[]){ int[] x = {35}; A a1; a1 = new A(); A a2 = new A(-5,-7); a1.methodB(1, x); a2.methodB(1, x); } } </pre>			

HOMEWORK

Task 1

Design the **Toy** class in such a way that it produces the following output

Driver Code	Expected Output
<pre>public class ToyTester{ public static void main(String[] args){ Toy t1 = new Toy("Car", 230); System.out.println("1====="); t1.updatePrice(250); System.out.println("2====="); System.out.println(t1.name); t1.showPrice(); System.out.println("3====="); Toy t2 = new Toy("Robot", 450); System.out.println("4====="); t2.updateName("Autobot"); System.out.println("5====="); System.out.println(t2.name); t2.showPrice(); } }</pre>	<pre>A new toy has been made! 1===== 2===== Car price: 250 Taka 3===== A new toy has been made! 4===== Changing old name: Robot new name: Autobot 5===== Autobot price: 450 Taka</pre>

Task 2

Complete the following **Cart** class to generate the given output from the tester code:

- A cart will have a cart number which will be assigned in *create_cart()* method.
- Each cart can hold up to 3 items (at max).
- Each cart must have two arrays to store items and their respective prices.
- The items inside a cart will be added in *addItem()* method only if the cart items do not exceed 3.
- The *giveDiscount()* method saves the discount given to that cart object and updates the price accordingly.

Driver Code	Output
<pre> public class CartTester{ public static void main(String [] args){ Cart c1 = new Cart (); Cart c2 = new Cart (); Cart c3 = new Cart (); c1.create_cart(1); c2.create_cart(2); c3.create_cart(3); System.out.println("====1===="); c1.addItem("Table", 3900.5); c1.addItem("Chair", 1400.76); c1.addItem(5400.87, "Television"); c1.addItem(5000.0, "Refrigerator"); System.out.println("====2===="); c2.addItem("Stove", 439.90); System.out.println("====3===="); c3.addItem("Chair", 1400.5); c3.addItem(3400.0, "Chair"); System.out.println("====4===="); c1.cartDetails(); System.out.println("====5===="); c2.cartDetails(); System.out.println("====6===="); c3.cartDetails(); c1.giveDiscount(10); System.out.println("====7===="); c1.cartDetails(); } } </pre>	<pre> ====1==== Table added to cart 1. You have 1 item(s) in your cart now. Chair added to cart 1. You have 2 item(s) in your cart now. Television added to cart 1. You have 3 item(s) in your cart now. You already have 3 items on your cart ====2==== Stove added to cart 2. You have 1 item(s) in your cart now. ====3==== Chair added to cart 3. You have 1 item(s) in your cart now. Chair added to cart 3. You have 2 item(s) in your cart now. ====4==== Your cart(c1) : Table - 3900.5 Chair - 1400.76 Television - 5400.87 Discount Applied: 0.0% Total price: 10702.130000000001 ====5==== Your cart(c2) : Stove - 439.9 Discount Applied: 0.0% Total price: 439.9 ====6==== Your cart(c3) : Chair - 1400.5 Chair - 3400.0 Discount Applied: 0.0% Total price: 4800.5 ====7==== Your cart(c1) : Table - 3900.5 Chair - 1400.76 Television - 5400.87 Discount Applied: 10.0% Total price: 9631.917000000001 </pre>

Task 3

Design the **Reader** class in such a way so that the following code provides the expected output.

- A reader will have a name, capacity to read and an array of books they are reading.
- The initial capacity of a reader will be 2. The initial name will be “New user”.

Driver Code	Expected Output
<pre>public class Reader_tester { public static void main(String[] args){ System.out.println("1 ====="); Reader r1 = new Reader("Messi"); Reader r2 = new Reader("Ronaldo", 3); System.out.println("2 ====="); r1.readerInfo(); System.out.println("3 ====="); r2.addBook("Java"); r2.addBook("Python"); r2.addBook("C++"); r2.readerInfo(); System.out.println("4 ====="); r1.addBook("C#"); r1.addBook("Rust"); r1.addBook("GoLang"); System.out.println("5 ====="); r2.addBook("Python"); System.out.println("6 ====="); r1.readerInfo(); System.out.println("7 ====="); r1.updateCapacity(3); System.out.println("8 ====="); r1.addBook("GoLang"); System.out.println("9 ====="); r1.readerInfo(); } }</pre>	<pre>1 ===== A new reader is created! A new reader is created! 2 ===== Name: Messi Capacity: 2 Books: No books added yet 3 ===== Name: Ronaldo Capacity: 3 Books: Book 1: Java Book 2: Python Book 3: C++ 4 ===== No more capacity 5 ===== No more capacity 6 ===== Name: Messi Capacity: 2 Books: Book 1: C# Book 2: Rust 7 ===== Capacity has changed to 3 8 ===== 9 ===== Name: Messi Capacity: 3 Books: Book 1: C# Book 2: Rust Book 3: GoLang</pre>

Task 4

You are building a tracker system that will keep track of a person's income and expenses.

- When the ***createTracker()*** method is invoked it sets the balance to 1.0 taka.
- The ***info()*** method **returns** a String with the trackers information.
- If the total balance becomes 0 after the ***expense()*** method is called it prints "You're broke!". Again if the available balance is less than the expense it prints "Not enough balance.". Otherwise the method prints "Balance updated" after updating the balance.
- The last expense and income history can be seen by using the ***history()*** method.

Driver Code	Output
<pre>public class Tester4{ public static void main(String[] args) { MoneyTracker tr1 = new MoneyTracker(); System.out.println(tr1.info()); tr1.createTracker("John"); System.out.println("1 ====="); System.out.println(tr1.info()); System.out.println("2 ====="); tr1.income(1000); System.out.println(tr1.info()); System.out.println("3 ====="); tr1.expense(800); tr1.expense(100); System.out.println(tr1.info()); System.out.println("4 ====="); tr1.showHistory(); System.out.println("5 ====="); tr1.expense(101); System.out.println("6 ====="); tr1.expense(200); System.out.println("7 ====="); tr1.income(200); tr1.showHistory(); System.out.println("8 ====="); } }</pre>	<pre>A new money tracker has been launched. Name: null Current Balance: 0.0 1 ===== Name: John Current Balance: 1.0 2 ===== Balance Updated! Name: John Current Balance: 1001.0 3 ===== Balance Updated. Balance Updated. Name: John Current Balance: 101.0 4 ===== Last added: 1000.0 Last spent: 100.0 5 ===== You're broke! 6 ===== Not enough balance. 7 ===== Balance Updated! Last added: 200.0 Last spent: 101.0 8 =====</pre>

Task 5

1	public class A{
2	public int x, y, z = 5;
3	public double p = 0.0;
4	public void methodA(int x, int m) {
5	this.x = methodB(this.x);
6	p = x + this.x % m * 3.0;
7	y = y + methodB(x++, this.x);
8	System.out.println(this.x + " " + x + y + " " + p) ;
9	}
10	public int methodB(int q, int n) {
11	int arr[] = {3,4,5};
12	arr[0] = arr[0] + this.x + n;
13	arr[1] = q + arr[1];
14	System.out.println(arr[0] + " " + arr[1] + " " + arr[2]) ;
15	return arr[1] + arr[2];
16	}
17	public int methodB(int y) {
18	if(y % 2 == 0) {
19	int temp = this.methodB(2, y);
20	return temp;
21	}
22	else{
23	return 4;
24	}
25	}
26	}

Driver Code	Output		
<pre> public class Tester11 { public static void main(String [] args){ A a1 = new A(); a1.methodA(2,3); a1.methodB(5,4); } } </pre>			

Task 6

1	public class Maze{
2	public int x;
3	public Maze(){
4	int x = 17;
5	this.methodB(x, -this.x);
6	}
7	public void methodA(){
8	int m = 0, x = 9;
9	m = methodB(m-3)+x;
10	this.x = ++x;
11	System.out.println(this.x+" "+m);
12	this.methodB(x,m);
13	System.out.println(x+" "+(m+this.x));
14	methodB(m);
15	}
16	public int methodB(int y){
17	x=y*y;
18	System.out.println(x+" "+y);
19	return x-11;
20	}
21	public void methodB(int z, int x){
22	z=z-2;
23	x=this.x-2*x;
24	System.out.println(z+" "+this.x);
25	}
26	}

[illegible]

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Driver Code	Output
<pre>public class CustomerTester { public static void main(String[] args) { Customer c1 = new Customer(); c1.createCustomer("John"); System.out.println("1====="); c1.showCart(); System.out.println("2====="); c1.addItem("Apple", 2); c1.addItem("Bread", 5); c1.addItem("Milk"); c1.addItem("Eggs", 2); System.out.println("3====="); c1.showCart(); System.out.println("4====="); c1.calculatePrice(); System.out.println("5====="); Customer c2 = new Customer(); c2.createCustomer("Jane"); c2.addItem("Apple", 2, "Orange", 5); c2.addItem("Chocolates", 15, "Bread", 5); c2.addItem("Milk", 3); System.out.println("6====="); c2.showCart(); System.out.println("7====="); c2.calculatePrice(); } }</pre>	<pre>1===== Customer: John 2===== Apple added to cart Orange added to cart Bread added to cart Milk added to cart Cart is full 3===== Customer: John Item: Apple Price: 2 Item: Orange Price: 5 Item: Bread Price: 5 Item: Milk Price: 3 4===== Total: 15 5===== Apple and Orange added to cart Chocolates and Bread added to cart Cart is full 6===== Customer: Jane Item: Apple Price: 2 Item: Orange Price: 5 Item: Chocolates Price: 15 Item: Bread Price: 5 7===== Total: 27</pre>

Task 2

Driver Code	Sample Output
<pre>public class CalculatorTester { public static void main(String[] args) { Calculator calc = new Calculator(); System.out.println("1====="); calc.add(10, 20); System.out.println("2====="); calc.add(5, 15, 25); System.out.println("3====="); calc.multiply(6, 7); System.out.println("4====="); calc.multiply(2, 3, 4); System.out.println("5====="); calc.multiply("Hello", 3); System.out.println("6====="); calc.multiply("Java", 5); } }</pre>	<pre>1===== 30 2===== 45 3===== 42 4===== 24 5===== Hello-Hello-Hello 6===== Java-Java-Java-Java-Java</pre>

Task 3

Driver Code	Sample Output
<pre>public class LibraryTest { public static void main(String[] args) { Book book1 = new Book(); book1.createBook("The Great Gatsby"); Book book2 = new Book(); book2.createBook("1984", "George Orwell"); Book book3 = new Book(); book3.createBook("To Kill a Mockingbird", "Harper Lee", "Fiction"); System.out.println(" ---Book Customization--- "); book1.customizeGenre("Classic"); book1.customizePages(180); book2.customizeGenre("Dystopian"); book2.customizePages(328); book3.customizePages(281); System.out.println(); System.out.println(" ---Library Inventory--- "); book1.displayDetails(); book2.displayDetails(); book3.displayDetails(); } }</pre>	<pre> ---Book Customization--- Updated genre of "The Great Gatsby" to Classic. Updated pages of "The Great Gatsby" to 180 pages. Updated genre of "1984" to Dystopian. Updated pages of "1984" to 328 pages. Updated pages of "To Kill a Mockingbird" to 281 pages. ---Library Inventory--- Title: The Great Gatsby, Author: Unknown, Genre: Classic, Pages: 180 Title: 1984, Author: George Orwell, Genre: Dystopian, Pages: 328 Title: To Kill a Mockingbird, Author: Harper Lee, Genre: Fiction, Pages: 281</pre>

Task 4

Driver Code	Sample Output
<pre>public class MovieManagerTest { public static void main(String[] args) { Movie inception = new Movie(); inception.setMovieDetails("Inception", "Christopher Nolan", 8.8); System.out.println("1====="); inception.addActors("Leonardo DiCaprio", "Joseph Gordon-Levitt"); inception.addActors("Ellen Page"); inception.showInfo(); System.out.println("2====="); Movie avengers = new Movie(); avengers.setMovieDetails("Avengers: Endgame", "Anthony Russo", 8.4); avengers.addActors("Robert Downey Jr.", "Chris Evans", "Scarlett Johansson"); avengers.showInfo(); System.out.println("3====="); Movie parasite = new Movie(); parasite.setMovieDetails("Parasite", "Bong Joon-ho"); parasite.addActors("Song Kang-ho", "Choi Woo-shik"); parasite.updateRating(8.6); parasite.showInfo(); System.out.println("4====="); parasite.updateRating(8.9); parasite.showInfo(); } }</pre>	<pre>1===== Added actor "Leonardo DiCaprio" to "Inception". Added actor "Joseph Gordon-Levitt" to "Inception". Added actor "Ellen Page" to "Inception". Title: Inception Director: Christopher Nolan Rating: 8.8 Actors: Leonardo DiCaprio, Joseph Gordon-Levitt, Ellen Page 2===== Added actor "Robert Downey Jr." to "Avengers: Endgame". Added actor "Chris Evans" to "Avengers: Endgame". Added actor "Scarlett Johansson" to "Avengers: Endgame". Title: Avengers: Endgame Director: Anthony Russo Rating: 8.4 Actors: Robert Downey Jr., Chris Evans, Scarlett Johansson 3===== Added actor "Song Kang-ho" to "Parasite". Added actor "Choi Woo-shik" to "Parasite". Updated rating of "Parasite" to 8.6 Title: Parasite Director: Bong Joon-ho Rating: 8.6 Actors: Song Kang-ho, Choi Woo-shik 4===== Updated rating of "Parasite" to 8.9 Title: Parasite Director: Bong Joon-ho Rating: 8.9 Actors: Song Kang-ho, Choi Woo-shik</pre>

Task 5

1	public class Test3 {
2	int x = 2, y = 4, sum = 3;
3	int arr[] = {x, y, sum};
4	public void methodA(int x) {
5	arr[0] += methodB(y, this.x) + methodC(x);
6	System.out.println(x + " " + this.x + " " + sum);
7	arr[1] += this.x * (++y) / (sum % x);
8	System.out.println(y + " " + sum + " " + this.x);
9	arr[2] += methodC(x) + methodB(this.x, sum);
10	System.out.println(arr[0] + " " + arr[1] + " " + arr[2]);
11	}
12	public int methodB(int q, int n) {
13	int arr2[] = {7, 8};
14	int a = (arr2[0]++) - q;
15	int b = (++arr2[1]) - n;
16	return a + b;
17	}
18	public int methodC(int z) {
19	z = sum + methodB(x, sum) - z;
20	return z/2;
21	}
22	}

<pre> public class Tester3{ public static void main(String [] args){ Test3 t3 = new Test3(); t3.methodA(7); } } </pre>	Outputs		

Lab Assignment 04



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Reference Passing, Multiclass Problem & Encapsulation
Number of Tasks:	13 (Classwork: 06, Homework: 07)

[Submit all the Coding Tasks (Homework: Task 1 to 4) in the Google Form shared on buX before the next lab. Submit the Tracing Tasks (Homework: Task 5 to 6) handwritten to your Lab Instructors at the beginning of the lab]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

Design the **Shop** class with necessary properties so that the following output is produced.

Driver Code	Output
<pre>public class ShopTester{ public static void main(String[] args){ Shop s1 = new Shop("Agora", 4); Shop s2 = new Shop(3); System.out.println("1====="); s1.info(); System.out.println("2====="); s2.info(); System.out.println("3====="); s1.addItem("KitKat", 25); s1.addItem("Hershey's", 100); s1.addItem("Dairy Milk", 150); s1.addItem("Feastables", 500); s1.addItem("Prime", 1200); System.out.println("4====="); s1.info(); System.out.println("5====="); s1.purchase("KitKat"); s1.purchase("Snickers"); System.out.println("6====="); s2.info(); System.out.println("7====="); s2.addItem(s1, "KitKat"); s2.addItem("Toblerone", 80); s2.addItem(s1, "Mars"); System.out.println("8====="); s2.info(); } }</pre>	<pre>Agora shop created! Mega shop created! 1===== Shop Name: Agora Item Details: No items in shop 2===== Shop Name: Mega Item Details: No items in shop 3===== Prime, could not be added 4===== Shop Name: Agora Item Details: 4 / 4 KitKat - 25 Tk Hershey's - 100 Tk Dairy Milk - 150 Tk Feastables - 500 Tk 5===== Purchase Complete! Snickers, is not available in this shop 6===== Shop Name: Mega Item Details: No items in shop 7===== Mars, not found in Agora! 8===== Shop Name: Mega Item Details: 2 / 3 KitKat - 25 Tk Toblerone - 80 Tk</pre>

Task 2

Write the **Teacher** and **Course** classes so that the TestTeacher class produces the outputs given.
Hint: A teacher can add a maximum of 3 courses.

[The attribute of Course class must be Private]

Driver Code	Output
<pre>public class TestTeacher{ public static void main(String [] args){ Teacher t1 = new Teacher("Matin Saad Abdullah","MSA"); Teacher t2 = new Teacher("Mumit Khan","MMK"); Teacher t3 = new Teacher("Sadia Hamid Kazi","SKZ"); Course c1 = new Course("CSE 110"); Course c2 = new Course("CSE 111"); Course c3 = new Course("CSE 220"); Course c4 = new Course("CSE 221"); Course c5 = new Course("CSE 230"); Course c6 = new Course("CSE 310"); Course c7 = new Course("CSE 320"); Course c8 = new Course("CSE 340"); t1.addCourse(c1); t1.addCourse(c2); t2.addCourse(c3); t2.addCourse(c4); t2.addCourse(c5); t3.addCourse(c6); t3.addCourse(c7); t3.addCourse(c8); System.out.println("1====="); t1.printDetail(); System.out.println("2====="); t2.printDetail(); System.out.println("3====="); t3.printDetail(); } }</pre>	<pre>A new teacher has been created A new teacher has been created A new teacher has been created 1===== Name: Matin Saad Abdullah Initial: MSA List of courses: CSE 110 CSE 111 2===== Name: Mumit Khan Initial: MMK List of courses: CSE 220 CSE 221 CSE 230 3===== Name: Sadia Hamid Kazi Initial: SKZ List of courses: CSE 310 CSE 320 CSE 340</pre>

Task 3

Design the **Team** and **Player** classes to get the output as shown.

Hints:

- Create an array in the **Team** class to store the **Player**'s object
- Use constructor overloading technique for **Team** class
- A team can add maximum 11 players
- All attributes of **Player** class must be kept **Private**.

Driver Code	Output
<pre>public class TeamTester { public static void main(String[] args) { Team b = new Team(); b.updateName("Bangladesh"); Player mashrafi = new Player("Mashrafi", 42, 100); b.addPlayer(mashrafi); Player tamim = new Player("Tamim", 35, 70); b.addPlayer(tamim); System.out.println("====="); b.printDetail(); System.out.println("====="); Team a = new Team("Australia"); Player ponting = new Player("Ponting", 50, 300); a.addPlayer(ponting); Player lee = new Player("Lee", 49, 200); a.addPlayer(lee); a.printDetail(); } }</pre>	<pre>===== Team: Bangladesh List of players: Name: Mashrafi Age: 42, Total Matches: 100 Name: Tamim Age: 35, Total Matches: 70 ===== Team: Australia List of players: Name: Ponting Age: 50, Total Matches: 300 Name: Lee Age: 49, Total Matches: 200</pre>

Task 4

Design the required class/es so that the following output is generated. Read the following description:

1. You may assume that to board a bus, a student must have the bus pass, and his/her location must match the route of the bus.
2. Additionally, the default maximum capacity of the bus is 2.
3. The location attribute of the **Student** class will be **private**

Driver Code	Output
<pre> public class BracuStudentTester { public static void main(String[] args) { BracuStudent st1 = new BracuStudent("Afif", "Mirpur"); System.out.println("1====="); BracuStudent st2 = new BracuStudent("Shanto", "Motijheel"); BracuStudent st3 = new BracuStudent("Taskin", "Mirpur"); st1.showDetails(); st2.showDetails(); System.out.println("2====="); st3.showDetails(); System.out.println("3====="); BracuBus bus1 = new BracuBus("Mirpur"); BracuBus bus2 = new BracuBus("Azimpur", 5); bus1.showDetails(); bus2.showDetails(); System.out.println("4====="); st2.collectPass(); st3.collectPass(); System.out.println("5====="); st2.showDetails(); st3.showDetails(); System.out.println("6====="); bus1.board(); System.out.println("7====="); bus1.board(st1, st2); System.out.println("8====="); st1.collectPass(); st2.setLocation("Mirpur"); st1.showDetails(); st2.showDetails(); System.out.println("9====="); bus1.board(st1); bus1.board(st2, st3); System.out.println("10====="); bus1.showDetails(); } } </pre>	<pre> 1===== Student Name: Afif Lives in Mirpur Have Bus Pass? false Student Name: Shanto Lives in Motijheel Have Bus Pass? false 2===== Student Name: Taskin Lives in Mirpur Have Bus Pass? false 3===== Bus Route: Mirpur Passenger Count: 0 (Max: 2) Passengers on Board: Bus Route: Azimpur Passenger Count: 0 (Max: 5) Passengers on Board: 4===== 5===== Student Name: Shanto Lives in Motijheel Have Bus Pass? true Student Name: Taskin Lives in Mirpur Have Bus Pass? true 6===== No passengers 7===== You don't have a bus pass! You got on the wrong bus! 8===== Student Name: Afif Lives in Mirpur Have Bus Pass? true Student Name: Shanto Lives in Mirpur Have Bus Pass? true 9===== Afif boarded the bus. Shanto boarded the bus. Bus is full! 10===== Bus Route: Mirpur Passenger Count: 2 (Max: 2) Passengers on Board: Afif Shanto </pre>

Task 5

1	public class Trace {
2	public int sum, temp;
3	public Trace(int sum, int temp){
4	this.sum = sum;
5	this.temp = temp;
6	}
7	}
8	class Quiz5{
9	public int sum = 11, x = -2, y = 16;
10	public Trace trace = null;
11	public Quiz5(Trace t){
12	trace = t;
13	int x = trace.temp + y;
14	sum = sum + (t.sum++) + y;
15	System.out.println(trace.sum + " " + sum + " " + x);
16	sum -= 10;
17	}
18	public void methodA(int sum, int temp){
19	sum = -3 + this.sum - trace.sum;
20	x = sum + 13 + y;
21	y = trace.temp + temp + sum;
22	this.sum = y + methodB(trace.temp, trace) + trace.temp;
23	System.out.println(sum + " " + y + " " + this.sum);
24	}
25	public int methodB(int x, Trace temp){
26	int sum = x + temp.sum + this.x;
27	temp.temp = sum + this.sum;
28	System.out.println(x + " " + temp.temp + " " + sum);
29	return sum;
30	}
31	}

<pre>Trace p = new Trace(3, 4); Quiz5 q = new Quiz5(p); q.methodA(4, 8); q.methodA(5, 10);</pre>	Output		

Task 6

1	public class TracingX {
2	public int x, y = 1;
3	public int metA(int y){
4	y += x + 3;
5	int temp = y + this.y;
6	if (temp % 2 == 0){
7	return temp;
8	}
9	TracingX t = new TracingX();
10	t.y = this.x - (++x) + t.x;
11	this.y = y + t.metA(t.x);
12	System.out.println(x + " " + y + " " + temp);
13	return temp + this.y;
14	}
15	}

Driver code:

```
public class TesterX {  
    public static void main(String[] args) {  
        TracingX t1 = new TracingX();  
        t1.y = t1.x = 5;  
        TracingX t2 = new TracingX();  
        t2.x = t1.metA(2);  
        t2.y = t2.metA(4);  
        System.out.println(t1.y + t1.x + " " + t2.x + " " + t2.y);  
    }  
}
```

Output:

HOMEWORK

Task 1

Design the **Triangle** Class that will produce the following output. We will consider both triangles to have the same sides if all sides are equal in the same orientation/sequence only.

Types of Triangle:

- Equilateral: When all sides in the same orientation are equal.
- Isosceles: When any two sides of a triangle in the same orientation are equal.
- Scalene: When all sides are of different lengths.

Driver Code	Output
<pre>public class TriangleTester{ public static void main(String args[]){ Triangle t1 = new Triangle(4, 4, 4); Triangle t2 = new Triangle(4, 5, 6); Triangle t3 = new Triangle(4, 5, 6); Triangle t4 = new Triangle(5, 4, 6); t1.triangleDetails(); System.out.println("-----1-----"); System.out.println(t1.printTriangleType()); System.out.println("-----2-----"); t3.triangleDetails(); System.out.println(t3.printTriangleType()); System.out.println("-----3-----"); t4.triangleDetails(); System.out.println(t4.printTriangleType()); System.out.println("-----4-----"); t2.compareTriangles(t3); System.out.println("-----5-----"); t1.compareTriangles(t2); System.out.println("-----6-----"); t1 = t2; t1.compareTriangles(t2); System.out.println("-----7-----"); t3.compareTriangles(t4); } }</pre>	<pre>Three sides of the triangle are: 4, 4, 4 Perimeter: 12 -----1----- This is an Equilateral Triangle. -----2----- Three sides of the triangle are: 4, 5, 6 Perimeter: 15 This is a Scalene Triangle. -----3----- Three sides of the triangle are: 5, 4, 6 Perimeter: 15 This is a Scalene Triangle. -----4----- Addresses are different but the sides of the triangles are equal. -----5----- Addresses, length of the sides and perimeter all are different. -----6----- These two triangle objects have the same address. -----7----- Only the perimeter of both triangles is equal.</pre>

Task 2

Design the **Player** class to obtain the following output from the driver code. You can assume, each player can defeat at most 5 villains.

Driver Code	Expected Output
<pre>public class PlayerTester { public static void main(String[] args) { Player ben = new Player("Ben", 10); System.out.println("=====1====="); ben.viewInfo(); System.out.println("=====2====="); ben.defeatVillain("Vilgax", 100); System.out.println("=====3====="); ben.defeatVillain("Yamcha", 10); System.out.println("=====4====="); ben.viewInfo(); System.out.println("=====5====="); ben.defeatVillain('8', "Vilgax", 100); System.out.println("=====6====="); ben.viewInfo(); System.out.println("=====7====="); Player kevin = new Player("Kevin"); System.out.println("=====8====="); kevin.viewInfo(); System.out.println("=====9====="); ben.defeatVillain(kevin); System.out.println("=====10====="); ben.viewInfo(); System.out.println("=====11====="); Player goku = new Player("Goku", 9000); System.out.println("=====12====="); ben.defeatVillain(goku); } }</pre>	<pre>Ben joined the game HP: 10 =====1===== Player Name: Ben Current HP: 10 =====2===== failed to defeat Vilgax =====3===== defeated Yamcha =====4===== Player Name: Ben Current HP: 20 Defeated: Yamcha, =====5===== HP with 8x boost: 160 defeated Vilgax =====6===== Player Name: Ben Current HP: 260 Defeated: Yamcha, Vilgax, =====7===== Kevin joined the game HP: 100 =====8===== Player Name: Kevin Current HP: 100 =====9===== defeated Kevin =====10===== Player Name: Ben Current HP: 360 Defeated: Yamcha, Vilgax, Kevin, =====11===== Goku joined the game HP: 9000 =====12===== failed to defeat Goku</pre>

Task 3

Design the **Student** and the **Connect** class so that the following output is produced.

Note:

- A student's email, password, and login status are null by default while creating an object of the Student class.
- The password and login status attributes of the Student class will be **private**
- Your code should satisfy the conditions mentioned in the output only.
- Connect class will have two instance variables: totalAdvisee and an array of Student type to store the student object. The array will be updated inside the advising() method only when the advising is successful.
- Connect can take at most 5 advisees.

Driver Code	Expected Output
<pre>public class ConnectTester { public static void main(String[] args) { Student rakib = new Student("Rakib", 12301455, "CSE"); Student roy = new Student("Roy", 12501345, "CS"); System.out.println("1*****"); Connect connectObj = new Connect(); System.out.println("2*****"); connectObj.login(rakib); System.out.println("3*****"); connectObj.advising(rakib); System.out.println("4*****"); rakib.email = "rakib@hotmail.com"; rakib.setPassword("1234"); System.out.println("5*****"); connectObj.login(rakib); System.out.println("6*****"); connectObj.advising(rakib); System.out.println("7*****"); connectObj.advising(rakib, "CSE110", "PHY111", "MAT110", "CSE260"); System.out.println("8*****"); connectObj.advising(rakib, "CSE110", "PHY111", "MAT110"); System.out.println("9*****"); connectObj.allAdviseeInfo(); System.out.println("10*****"); roy.email = "roy@hotmail.com"; roy.setPassword("abcd"); connectObj.login(roy); System.out.println("11*****"); connectObj.advising(roy, "CSE110", "ENG101", "PHY112"); System.out.println("12*****"); connectObj.allAdviseeInfo(); } }</pre>	<pre>Student object is created Student object is created 1***** Connect is ready to use! 2***** Email and password need to be set. 3***** Please login to advise courses! 4***** 5***** Login successful 6***** You haven't selected any courses. 7***** You need special approval to take more than 3 courses. 8***** Advising successful! 9***** Total Advisee: 1 Name: Rakib ID: 12301455 Department: CSE Advised Courses: CSE110 PHY111 MAT110 ===== 10***** Login successful 11***** Advising successful! 12***** Total Advisee: 2 Name: Rakib ID: 12301455 Department: CSE Advised Courses: CSE110 PHY111 MAT110 ===== Name: Roy ID: 12501345 Department: CS Advised Courses: CSE110 ENG101 PHY112 =====</pre>

Task 4

Spaceship: This class represents a spaceship. Each spaceship has a **name** and a **capacity** (the maximum weight it can carry).

Cargo: This class represents a piece of cargo. Each cargo item has a **name** and a **weight**. Both attributes should be **private** which means they cannot be accessed directly from outside of the class.

A **Spaceship** contains **Cargo**. That means each spaceship can carry multiple cargo items, but the total weight of the cargo cannot exceed the spaceship's capacity. Also, the maximum number of cargo items is 100. Your task is to design the **Spaceship** and **Cargo** class with necessary properties so that the given output is produced for the provided driver code.

Driver Code	Output
<pre>public class ShipTester { public static void main(String[] args) { Spaceship falcon = new Spaceship("Falcon", 50000); Spaceship apollo = new Spaceship("Apollo", 100000); Spaceship enterprise = new Spaceship("Enterprise",220000); System.out.println("1.====="); Cargo gold = new Cargo("Gold", 20000); Cargo platinum = new Cargo("Platinum", 25000); Cargo dilithium = new Cargo("Dilithium", 50000); Cargo trilithium = new Cargo("Trilithium", 70000); Cargo neutronium = new Cargo("Neutronium", 80000); System.out.println("2.====="); falcon.loadCargo(gold); falcon.loadCargo(platinum); falcon.displayDetails(); System.out.println("3.====="); apollo.loadCargo(gold); apollo.displayDetails(); System.out.println("4.====="); falcon.loadCargo(neutronium); System.out.println("5.====="); enterprise.loadCargo(dilithium); enterprise.loadCargo(trilithium); enterprise.loadCargo(neutronium); enterprise.displayDetails(); } }</pre>	<pre>1.===== 2.===== Spaceship Name: Falcon Capacity: 50000 Current Cargo Weight: 45000 Cargo:Gold Platinum 3.===== Spaceship Name: Apollo Capacity: 100000 Current Cargo Weight: 20000 Cargo:Gold 4.===== Warning: Unable to load Neutronium inside Falcon. Exceeds capacity by 75000. 5.===== Spaceship Name: Enterprise Capacity: 220000 Current Cargo Weight: 200000 Cargo:Dilithium Trilithium Neutronium</pre>

Task 5

1	public class TestX{
2	public int result = 17, i, n = 7, m = 5;
3	public int [] num = {6,8,11};
4	public void compute(int [] nums){
5	n = nums[0] + this.m;
6	this.result += nums[i];
7	TestX temp = new TestX();
8	boolean check = !(temp.modify(temp, num));
9	if (check) {
10	m = nums[1] + temp.result;
11	} else {
12	m = result - nums[3];
13	}
14	System.out.println(m + " " + n + " " + result);
15	}
16	public boolean modify(TestX obj, int[] arr){
17	arr[1]++;
18	i=i+1;
19	obj.m = this.result + arr[i];
20	obj.result = obj.m - obj.n + n;
21	System.out.println(obj.m + " " + obj.n + " " + result);
22	return arr[1] % 2 == 0;
23	}
24	public boolean modify(TestX obj){
25	num[i]++;
26	i=i+1;
27	obj.m = this.result + num[i];
28	obj.result = num[2] - obj.n + n;
29	System.out.println(obj.m + " " + obj.n + " " + result);
30	return num[1] % 2 != 0;
31	}
32	}

<pre> public class tester12{ public static void main(String[] args){ TestX tx = new TestX(); int[] data = {4, 6, 2, 3, 5}; tx.compute(data); tx.modify(tx); } } </pre>	Outputs		

Task 6

1	public class msgClass{
2	public int content;
3	}
4	class FinalT5A{
5	public int sum = 2, y = 1, x = 1;
6	public void methodA(){
7	int x = 6, y = 0;
8	msgClass myMsg = new msgClass();
9	myMsg.content = this.x;
10	x = x + myMsg.content;
11	this.y = this.y + methodB(myMsg, myMsg.content);
12	System.out.println(x + " " + this.y+ " " + sum);
13	y = this.y/2 + this.x;
14	x = y + sum/2;
15	sum = x + y + myMsg.content;
16	System.out.println(x + " " + y+ " " + sum);
17	}
18	public int methodB(msgClass mg2, int mg1){
19	int x = 0;
20	y = y + mg2.content;
21	mg2.content = y + mg1;
22	x = this.x + 3 + mg1;
23	sum = sum + x + y;
24	System.out.println(this.x + " " + this.y+ " " + sum);
25	mg2.content = sum - mg1 ;
26	return sum;
27	}
28	}

DRIVER CODE	OUTPUTS		
<pre> public class Tester5{ public static void main(String args []){ FinalT5A fT5A = new FinalT5A(); fT5A.methodA(); } } </pre>			

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Design the Company and Employee classes so that the Tester1 class produces the given outputs.

Restriction: Company class can't have more than 1 array and all attributes of Employee class must be Private.

Driver Code	Output
<pre>public class Tester1{ public static void main(String args[]){ Employee e1 = new Employee(); Employee e2 = new Employee("Alif", 34, "Fulltime"); System.out.println("1-----"); Company c1 = new Company(); c1.details(); System.out.println("2-----"); Employee e3 = new Employee("Akter", 36, "Part-time"); Employee e4 = new Employee("Ria", 38, "Fulltime"); System.out.println("3-----"); c1.addEmployee(e2); c1.addEmployee(e3); System.out.println("4-----"); c1.details(); System.out.println("5-----"); c1.addEmployee(e4); c1.addEmployee(e1); System.out.println("6-----"); c1.details(); System.out.println("7-----"); c1.removeEmployee(e4); System.out.println("6-----"); c1.details(); } }</pre>	<pre>A default employee has been created 1----- Company Name: ABC Company Total Employee: 0 Fulltime Employees: Part-Time Employees: 2----- 3----- Alif has joined the company Akter has joined the company 4----- Company Name: ABC Company Total Employee: 2 Fulltime Employees: Name: Alif, ID: 34 Part-Time Employees: Name: Akter, ID: 36 5----- Ria has joined the company No more vacancy 6----- Company Name: ABC Company Total Employee: 3 Fulltime Employees: Name: Alif, ID: 34 Name: Ria, ID: 38 Part-Time Employees: Name: Akter, ID: 36 7----- Ria has left the company 6----- Company Name: ABC Company Total Employee: 2 Fulltime Employees: Name: Alif, ID: 34 Part-Time Employees: Name: Akter, ID: 36</pre>

Task 2

Write “**Student**” class to show the following expected outputs

Note:

- ❖ A student can't take any course until the CGPA is set.
- ❖ A student cannot take more than 4 courses.
- ❖ A student with CGPA below 3 cannot take more than 3 courses.

Driver Code	Expected Output
<pre>public class StudentDriver { public static void main(String[] args){ Student student1 = new Student(12345678); System.out.println("1-----"); student1.addCourse("CSE110"); System.out.println("2-----"); student1.storeCG(2.5); student1.addCourse("CSE110"); student1.addCourse("ENG101"); student1.showAdvisee(); System.out.println("3-----"); student1.removeAllCourse(); student1.showAdvisee(); System.out.println("4-----"); student1.storeID(54652365); String[] courses = {"SOC101","CSE111","ENG102"}; student1.addCourse(courses); student1.showAdvisee(); System.out.println("5-----"); student1.addCourse("CSE230"); student1.showAdvisee(); System.out.println("6-----"); Student student2 = new Student(975738383,3.7); System.out.println("7-----"); String[] courses2 = {"CSE220","PHY112","MAT120","BUS101","CHN101"}; student2.addCourse(courses2); student2.showAdvisee(); } }</pre>	<pre>A student with ID 12345678 has been created. 1----- Failed to add CSE110 Set CG first 2----- Student ID: 12345678, CGPA: 2.5 Added courses are: CSE110 ENG101 3----- Student ID: 12345678, CGPA: 2.5 No courses added. 4----- Student ID: 54652365, CGPA: 2.5 Added courses are: SOC101 CSE111 ENG102 5----- Failed to add CSE230 CG is low. Can't add more than 3 courses. Student ID: 54652365, CGPA: 2.5 Added courses are: SOC101 CSE111 ENG102 6----- A student with ID 975738383 and cgpa 3.7 has been created. 7----- Failed to add CHN101 Maximum 4 courses allowed. Student ID: 975738383, CGPA: 3.7 Added courses are: CSE220 PHY112 MAT120 BUS101</pre>

Task 3

Design the **necessary** classes for the NetFlix system with the necessary properties to produce the given output for the provided driver code.

Driver code	Output
<pre>public class NetflixAccount{ public static void main(String[] args){ Movie m1 = new Movie("Avengers", "Sci-fi", "1:59:08"); Movie m2 = new Movie("Lord of the Rings", "Fantasy", "2:58:05"); Movie m3 = new Movie("Final Destination","Horror"); Movie m4 = new Movie("Fast and Furious","Action", "2:05:04"); System.out.println("====="); NetflixUser user1 = new NetflixUser("Martin Freeman", "Free"); user1.addToFavourites(m1); user1.showFavourites(); user1.upgradePlan(); user1.addToFavourites(m1); user1.showFavourites(); System.out.println("====="); NetflixUser user2 = new NetflixUser("Vin Diesel", "Premium"); user2.addToFavourites(m1); user2.addToFavourites(m2); user2.addToFavourites(m3); user2.addToFavourites(m4); user2.showFavourites(); System.out.println("====="); NetflixUser user3 = new NetflixUser("Chris Evans"); user3.addToFavourites(m1); user3.upgradePlan(); user3.addToFavourites(m2); user3.addToFavourites(m3); user1.upgradePlan(); } }</pre>	<pre>===== New user account created! Cannot add movies to favourites. Please upgrade to Netflix Premium. No available favourites. Please upgrade to Netflix Premium. Welcome to Netflix Premium! Movie added to favourites. Showing Martin Freeman's Favourites: Avengers, Sci-fi, Duration: 1:59:08 ===== New user account created! Movie added to favourites. Movie added to favourites. Movie added to favourites. Cannot add new movies. Favourites list is full. Showing Vin Diesel's Favourites: Avengers, Sci-fi, Duration: 1:59:08 Lord of the Rings, Fantasy, Duration: 2:58:05 Final Destination, Horror, Duration: Unknown ===== New user account created! Cannot add movies to favourites. Please upgrade to Netflix Premium. Welcome to Netflix Premium! Movie added to favourites. Movie added to favourites. You already have Netflix Premium!</pre>

Task 4

Please write the **Student** and **Department** class with the necessary properties so that the provided driver code generates the output given below. The id of the Student class will be **private**. For simplicity, assume that a department can add a maximum of 5 students.

Driver Code	Output
<pre>public class Tester1 { public static void main(String[] args) { Student s1 = new Student("Akib", 10, 3.29); Student s2 = new Student("Reza", 15, 3.45); Student s3 = new Student("Kabir", 20,4.0); System.out.println("1====="); Department cse = new Department("CSE"); cse.findStudent(-100); System.out.println("2====="); cse.addStudent(s1, s2, s3); System.out.println("3====="); cse.details(); System.out.println("4====="); cse.findStudent(15); System.out.println("5====="); Student s4 = new Student("Nakib", 15,3.22); cse.addStudent(s4); System.out.println("6====="); s4.setId(25); cse.addStudent(s4); System.out.println("7====="); cse.details(); System.out.println("8====="); Student s5 = new Student("Sakib", 30,2.29); cse.addStudent(s5); System.out.println("9====="); cse.details(); } }</pre>	<pre>1===== Student with this ID doesn't exist, Please give a valid ID 2===== Welcome to CSE department, Akib Welcome to CSE department, Reza Welcome to CSE department, Kabir 3===== Department Name: CSE Number of student:3 Details of the students: Student name: Akib, ID: 10, cgpa: 3.29 Student name: Reza, ID: 15, cgpa: 3.45 Student name: Kabir, ID: 20, cgpa: 4.0 4===== Student info: Student Name: Reza ID: 15 CGPA: 3.45 5===== Student with the same ID already exists, Please try with another ID 6===== Welcome to CSE department, Nakib 7===== Department Name: CSE Number of student:4 Details of the students: Student name: Akib, ID: 10, cgpa: 3.29 Student name: Reza, ID: 15, cgpa: 3.45 Student name: Kabir, ID: 20, cgpa: 4.0 Student name: Nakib, ID: 25, cgpa: 3.22 8===== Welcome to CSE department, Sakib 9===== Department Name: CSE Number of student:5 Details of the students: Student name: Akib, ID: 10, cgpa: 3.29 Student name: Reza, ID: 15, cgpa: 3.45 Student name: Kabir, ID: 20, cgpa: 4.0 Student name: Nakib, ID: 25, cgpa: 3.22 Student name: Sakib, ID: 30, cgpa: 2.29</pre>

Task 5

1	public class QuizA {
2	public int x, y;
3	public int sum = 1;
4	public QuizA(int x, int y) {
5	this.x = y;
6	this.y = x;
7	}
8	public void methodA() {
9	int x = 3;
10	y = this.y + x;
11	QuizA exam = new QuizA(5, 11);
12	exam.sum = x;
13	exam.y = this.y;
14	x = this.x + x + exam.sum;
15	this.y = this.sum + methodB(exam.sum, exam);
16	System.out.println(exam.x + " " + this.y + " " + sum);
17	sum = x % 2 + this.x;
18	y = x + y + exam.sum;
19	System.out.println(x + " " + y + " " + sum);
20	}
21	public int methodB(int x1, QuizA x2) {
22	int y = 0;
23	y = this.y + x2.sum;
24	x2.sum = x1 + x2.x;
25	sum = sum + x + y;
26	System.out.println(this.x + " " + this.y + " " + sum);
27	return x2.sum;
28	}
29	}

Driver Code	Output		
<pre> public class QuizTesterA{ public static void main(String []args){ QuizA q1 = new QuizA(3,4); q1.methodA(); } } </pre>			

Task 6

1	public class Trace{
2	public int x, y = 5, temp = -4, sum = 3;
3	public Trace(){
4	y = temp + 3 ;
5	sum = 3 + temp + 2;
6	temp -= 2;
7	}
8	public Trace(Trace t){
9	sum = t.sum++;
10	x = t.x + 2;
11	t.methodB(2,3);
12	}
13	public void methodA(int m, int n){
14	int x = 2;
15	y = y + m + (temp++);
16	x = x + 5 + n;
17	sum = sum + x + y;
18	System.out.println(x + " " + y+ " " + sum);
19	}
20	public void methodB(int m, int n){
21	int y = 0;
22	y = y + this.y;
23	x = this.y + 2 + temp;
24	methodA(x, y);
25	sum = x + y + sum;
26	System.out.println(x + " " + y+ " " + sum);
27	}
28	}

<pre> public class TraceTester { public static void main(String args []){ Trace t1 = new Trace(); Trace t2 = new Trace(t1); t1.methodA(1, 2); t2.methodB(3, 2); } } </pre>	Outputs		

Task 7

1	public class A {
2	public int x, y = 2, sum = 5;
3	public A(){
4	x = sum - y;
5	this.methodB(1);
6	}
7	public A(int x){
8	this.x = x;
9	}
10	public void methodB(int y) {
11	int temp = (y++) + (++this.y);
12	sum = y + temp - ++this.x;
13	System.out.println(y + " " + temp + " " + (sum++));
14	this.y = (++temp) + methodC(temp, y) + x;
15	}
16	public void methodA(int temp, int x) {
17	A a2 = new A(6);
18	a2.sum = this.methodC(a2);
19	System.out.println(this.x + " " + a2.sum + " " + y);
20	this.sum = a2.methodC(this);
21	System.out.println(a2.x + " " + this.sum + " " + a2.y);
22	}
23	public int methodC(A a) {
24	return this.x + a.y + a.sum;
25	}
26	public int methodC(int sum, int y) {
27	y = (this.x++) + sum + 3;
28	return y;
29	}
30	}

<pre> public class TesterA{ public static void main(String [] args) { A a1 = new A(); a1.methodA(4, 3); } } </pre>	Outputs		

Lab Assignment 05



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Encapsulation, Static variable and Static Method
Number of Tasks:	13 (Classwork: 06, Homework: 07)

[Submit all the Coding Tasks (Homework: Task 1 to 5) in the Google Form shared on buX before the next lab. Submit the Tracing Tasks (Homework: Task 6 to 7) handwritten to your Lab Instructors at the beginning of the lab]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

Design the **Account** class such that it produces the following output.

- Each Account will have a name, address and a balance variable.
- The balance must be kept private.
- Use Encapsulation to access and modify balance.

After finishing the class design to get the following output, try to directly print the balance variable. Can you print the private balance variable in the tester code?

Tester Code	Output
<pre>public class AccountTester { public static void main(String[] args) { System.out.print("All accounts belong to "); System.out.println(Account.bankName); System.out.println("Accounts created: " + Account.count); System.out.print("Total Money stored "); System.out.println(Account.totalBalance + " taka"); System.out.println("1-----"); Account a1 = new Account("Abir", "Dhanmondi"); System.out.println("2-----"); a1.printInfo(); System.out.println("3-----"); Account a2 = new Account("Mira", "Mirpur"); a2.printInfo(); System.out.println("4-----"); a1.setBalance(1000); System.out.println("5-----"); a1.printInfo(); a2.setBalance(2000); System.out.println("6-----"); System.out.println(a2.getBalance()); System.out.println("7-----"); a2.setBalance(500); System.out.println("8-----"); a2.printInfo(); System.out.println("9-----"); System.out.print("All accounts belong to "); System.out.println(Account.bankName); System.out.println("Accounts created: " + Account.count); System.out.print("Total Money stored "); System.out.println(Account.totalBalance + " taka"); } }</pre>	<pre>All accounts belong to Badda Bank Accounts created: 0 Total Money stored 0.0 taka 1----- 2----- Name: Abir, address: Dhanmondi Balance: 0.0 Taka 3----- Name: Mira, address: Mirpur Balance: 0.0 Taka 4----- 5----- Name: Abir, address: Dhanmondi Balance: 1000.0 Taka 6----- 2000.0 7----- 8----- Name: Mira, address: Mirpur Balance: 2500.0 Taka 9----- All accounts belong to Badda Bank Accounts created: 2 Total Money stored 3500.0 taka</pre>

Task 2

Design the **Student** class such that it produces the following output.

Note: A Student needs at least 50 marks to pass.

Tester Code	Output
<pre>public class MarksTester { public static void main(String[] args) { System.out.println("Total Students: " + Student.total_students); System.out.println("Average Marks: " + Student.averageMarks()); System.out.println("-----1-----"); Student mike = new Student("Mike"); mike.setCodingMarks(35); mike.setTracingMarks(10); mike.individualDetail(); System.out.println("-----2-----"); System.out.println("Total Students: " + Student.total_students); System.out.println("Average Marks: " + Student.averageMarks()); System.out.println("-----3-----"); Student eleven = new Student("Eleven", 70, 20); eleven.individualDetail(); System.out.println("-----4-----"); Student will = new Student("Will"); will.setCodingMarks(51); will.individualDetail(); System.out.println("-----5-----"); System.out.println("Total Students: " + Student.total_students); System.out.println("Average Marks: " + Student.averageMarks()); } }</pre>	<pre>Total Students: 0 Average Marks: 0.0 -----1----- Name: Mike ID: 1 Coding Marks: 35 Tracing Marks: 10 Mike has failed with 45 marks -----2----- Total Students: 1 Average Marks: 45.0 -----3----- Name: Eleven ID: 2 Coding Marks: 70 Tracing Marks: 20 Eleven has passed with 90 marks -----4----- Name: Will ID: 3 Coding Marks: 51 Tracing Marks: 0 Will has passed with 51 marks -----5----- Total Students: 3 Average Marks: 62.0</pre>

Task 3

From the given Tester Code write the **Trainer** class that uses the **Pokemon** class to produce the given output.

- YOU CANNOT change the given Pokemon class.
- Each Trainer can catch at max 7 pokemons.

Tester Code	Output
<pre>public class PokemonTrainerTester { public static void main(String[] args) { Pokemon pikachu = new Pokemon("Pikachu", 65.0); Pokemon caterpie = new Pokemon("Caterpie", 45.0); Pokemon squirtle = new Pokemon("Squirtle", 70.0); Pokemon eevee = new Pokemon("Eevee", 60.0); System.out.println("1-----"); Trainer ash = new Trainer("Ash"); System.out.println("2-----"); ash.catchPokemon(pikachu); System.out.println("3-----"); ash.catchPokemon(caterpie); ash.catchPokemon(squirtle); System.out.println("4-----"); ash.viewPokeDex(); System.out.println("5-----"); Trainer gary = new Trainer("Gary"); System.out.println("6-----"); gary.catchPokemon(squirtle); gary.catchPokemon(eevee); System.out.println("7-----"); gary.viewPokeDex(); System.out.println("8-----"); Trainer.battle(ash, gary); } }</pre>	<pre>1----- Trainer ID: 1, Name: Ash - created 2----- Ash caught: Pikachu 3----- Ash caught: Caterpie Ash caught: Squirtle 4----- Ash's Trainer code: T1 Pikachu: 65.0 points Caterpie: 45.0 points Squirtle: 70.0 points Average of HP: 60.0 5----- Trainer ID: 2, Name: Gary - created 6----- Gary caught: Squirtle Gary caught: Eevee 7----- Gary's Trainer code: T2 Squirtle: 70.0 points Eevee: 60.0 points Average of HP: 65.0 8----- Battle Info Ash's HP average: 60.0 Gary's HP average: 65.0 Gary wins</pre>
Given Class	
<pre>public class Pokemon { private String name; private double hp; public Pokemon(String n, double hp) { this.name = n; this.hp = hp; } public String getName() { return name; } public double getHP() { return hp; } }</pre>	

Task 4

Suppose you have opened a new library, from where your friends can borrow books. Initially you have bought 3 books (Pather Panchali, Durgesh Nandini & Anandmath) each of 3 copies only. Design the **Borrower** class in such a way that the following code provides the expected output.

- You are given the arrays **book_count** and **book_name** to keep track of the number of books available. For simplicity, assume that there will be no other books in the library.
- You must reuse the **remainingBooks()** method when needed.

Given Code	Expected Output
<pre>public class BorrowerTester{ public static void main(String args[]){ Borrower.bookStatus(); System.out.println("*****1*****"); Borrower b1 = new Borrower("Nabila"); b1.borrowBook("Pather Panchali"); b1.borrowBook("Anandmath"); b1.borrowerDetails(); System.out.println("*****2*****"); Borrower b2 = new Borrower("Sadia"); b2.borrowBook("Anandmath"); b2.borrowBook("Durgesh Nandini"); b2.borrowBook("Pather Panchali"); b2.borrowerDetails(); System.out.println("*****3*****"); System.out.println(Borrower.remainingBooks("Anandmath")+ copies of Anandmath is remaining."); System.out.println("*****4*****"); Borrower b3 = new Borrower("Anika"); b3.borrowBook("Anandmath"); Borrower.bookStatus(); System.out.println("*****5*****"); Borrower b4 = new Borrower("Oishi"); b4.borrowBook("Anandmath"); b4.borrowBook("Durgesh Nandini"); b4.borrowerDetails(); } } public class Borrower{ public static int book_count[] = {3, 3, 3}; public static String book_name[] = {"Pather Panchali", "Durgesh Nandini", "Anandmath"}; // Your Code here }</pre>	<pre>Available Books: Pather Panchali: 3 Durgesh Nandini: 3 Anandmath: 3 *****1***** Name: Nabila Books Borrowed: Pather Panchali Anandmath *****2***** Name: Sadia Books Borrowed: Anandmath Durgesh Nandini Pather Panchali *****3***** 1 copies of Anandmath is remaining. *****4***** Available Books: Pather Panchali: 1 Durgesh Nandini: 2 Anandmath: 0 *****5***** This book is not available. Name: Oishi Books Borrowed: Durgesh Nandini</pre>

Task 5

1	public class B{
2	public static int x;
3	public int y = 4;
4	public int temp = -5;
5	public int sum = 2;
6	public B(){
7	y = temp + 3 ;
8	sum = 3 + temp + 3;
9	temp-=2;
10	}
11	public B(B b){
12	sum = b.sum;
13	x = b.x;
14	b.methodB(1,3);
15	}
16	public void methodA(int m, int n){
17	int x = 2;
18	y = y + m + (temp++);
19	x = x + 7 + n;
20	sum = sum + x + y;
21	System.out.println(x + " " + y+ " " + sum);
22	}
23	public void methodB(int m, int n){
24	int y = 0;
25	y = y + this.y;
26	x = this.y + 3 + temp;
27	methodA(x, y);
28	sum = x + y + sum;
29	System.out.println(x + " " + y+ " " + sum);
30	}
31	}

Consider the following driver code and find the output.

B b1 = new B();	x	y	sum
B b2 = new B(b1);			
b1.methodA(3, 2);			
b2.methodB(1, 2);			

Task 6

1	class Trace6CW {
2	public static int temp = 10;
3	public int sum = 0;
4	public static int[] y = {2, 5};
5	public Trace6CW() {
6	temp -= 2;
7	sum = temp + y[0];
8	y[1] = sum - temp;
9	}
10	public void methodA(int m, int n) {
11	int temp = 0;
12	temp = Trace6CW.temp + m;
13	this.sum = this.sum + temp + Trace6CW.y[1];
14	Trace6CW.y[0] = this.sum - n;
15	System.out.println(this.sum + " " + temp + " " + Trace6CW.y[0]);
16	}
17	public static void methodB(Trace6CW s1, int m) {
18	int sum = 5;
19	y[0] = s1.sum + sum;
20	s1.sum = temp + y[1] + m;
21	System.out.println(s1.sum + " " + y[0] + " " + temp);
22	}
23	}

<pre> class Test5CW { public static void main(String[] args) { Trace6CW s1 = new Trace6CW(); Trace6CW s2 = new Trace6CW(); s1.methodA(3, 2); Trace6CW.methodB(s2, 4); s2.methodA(1, 1); } } </pre>	OUTPUT		

HOMEWORK

Task 1

Design the **Product** class such that it produces the following output.

Tester Code	Output
<pre>public class ProductTester{ public static void main(String[] args) { Product p1 = new Product("Table", 10); Product p2 = new Product("Chair", 15); Product p3 = new Product("Sofa", 20); Product p4 = new Product("Divan", 8); System.out.println("-----1-----"); Product.displayProducts(); System.out.println("-----2-----"); Product.buy("Chair", 5); System.out.println("-----3-----"); Product.displayProducts(); System.out.println("-----4-----"); Product.buy("Sofa", 25); System.out.println("-----5-----"); Product.displayProducts(); System.out.println("-----6-----"); Product.buy("Bed", 10); } }</pre>	<pre>Stored: Table Stored: Chair Stored: Sofa Storage is full! Cannot add Divan -----1----- === Stored Products === Table - Qty: 10 Chair - Qty: 15 Sofa - Qty: 20 -----2----- Product Sold -----3----- === Stored Products === Table - Qty: 10 Chair - Qty: 10 Sofa - Qty: 20 -----4----- Quantity low -----5----- === Stored Products === Table - Qty: 10 Chair - Qty: 10 Sofa - Qty: 20 -----6----- Product not found</pre>

Task 2

Design the **Character** class such that it produces the following output.

Tester Code	Output
<pre>public class WeirderStuffTester { public static void main(String[] args) { Character.printStats(); System.out.println("-----1-----"); Character twelve = new Character("Twelve", "Kid", 100); twelve.printDetails(); System.out.println("-----2-----"); Character spike = new Character("Spike", "Kid", 50); spike.printDetails(); System.out.println("-----3-----"); Character.printStats(); System.out.println("-----4-----"); Character reeve = new Character("Reeve", 70); reeve.printDetails(); System.out.println("-----5-----"); Character chopper = new Character("Chopper", "Adult", 120); chopper.printDetails(); System.out.println("-----6-----"); Character.printStats(); } }</pre>	<pre>Total Characters: 0 Kids: 0 Teens: 0 Adults: 0 Average Health: 0 Strongest Character: None -----1----- ID: 1, Name: Twelve Group: Kid Health: 100 -----2----- ID: 2, Name: Spike Group: Kid Health: 50 -----3----- Total Characters: 2 Kids: 2 Teens: 0 Adults: 0 Average Health: 75.0 Strongest Character: Twelve (Health 100) -----4----- ID: 3, Name: Reeve Group: Teen Health: 70 -----5----- ID: 4, Name: Chopper Group: Adult Health: 120 -----6----- Total Characters: 4 Kids: 2 Teens: 1 Adults: 1 Average Health: 85.0 Strongest Character: Chopper (Health 120)</pre>

Task 3

Design the **Artifact** class where all the attributes kept inside the class are classified i.e. private.

The “Vault” can only store a maximum of 4 artifacts. The power of the artifacts are calculated as:

For artifact with,

- Even length name => Summation of the ascii values of all characters in even index of name.
- Odd length name => Summation of the ascii values of all characters in odd index of name.

Tester Code	Output
<pre>public class TesterArtifact{ public static void main(String[] args) { Artifact a = new Artifact("Phone Microwave", "Kurusu"); System.out.println("-----1-----"); Artifact.AddtoVault(a); Artifact.AddtoVault(new Artifact("D-Mail Capsule", "Mayuri")); System.out.println("-----2-----"); Artifact c = new Artifact("C204 Chip"); Artifact d = new Artifact("Divergence Meter"); Artifact e = new Artifact("M4A2 Robot", "Okabe"); Artifact.AddtoVault(c); Artifact.AddtoVault(d); Artifact.AddtoVault(e); System.out.println("-----3-----"); Artifact.labReport(); System.out.println("-----4-----"); System.out.println("Power of "+c.GetName()+" is "+c.CalcPower()); System.out.println("-----5-----"); System.out.println("Strongest Artifact: "+Artifact.strongest()); System.out.println("-----6-----"); a.revealArtifact(); System.out.println("-----7-----"); a.changeName("Banana Microwave"); System.out.println("-----8-----"); Artifact.labReport(); System.out.println("-----9-----"); System.out.println("Strongest Artifact: "+Artifact.strongest()); } }</pre>	<pre>-----1----- Kurusu added Phone Microwave successfully to the vault. Mayuri added D-Mail Capsule successfully to the vault. -----2----- Okabe added C204 Chip successfully to the vault. Okabe added Divergence Meter successfully to the vault. !!Okabe unsuccessful in adding artifact to the vault!! -----3----- === Future Gadget Lab === Phone Microwave added by Kurisu has power of 702. D-Mail Capsule added by Mayuri has power of 602. C204 Chip added by Okabe has power of 274. Divergence Meter added by Okabe has power of 734. -----4----- Power of C204 Chip is 274 -----5----- Strongest Artifact: Divergence Meter -----6----- Phone Microwave added by Kurisu has power of 702. -----7----- Name changed and power recalculated. -----8----- === Future Gadget Lab === Banana Microwave added by Kurisu has power of 774. D-Mail Capsule added by Mayuri has power of 602. C204 Chip added by Okabe has power of 274. Divergence Meter added by Okabe has power of 734. -----9----- Strongest Artifact: Banana Microwave</pre>

Task 4

Design the **AnimalKeepers** class with the following requirements:

- The Animal Keepers have private IDs starting from 101.
- The Safari has an array named, Animals = {"Lion", "Tiger", "Seal", "Gorilla", "Deer"}
- Only one task is assigned per animal and so the tasks are overridden when reassigned for the same animal.

[Hint: You can call the static method printTasks() from inside details()]

Tester Code	Output
<pre>import java.util.Arrays; public class KeeperTester { public static void main(String[] args) { Animalkeepers.details(); System.out.println("-----1-----"); System.out.println(Arrays.toString(Animalkeepers.Animals)); System.out.println("-----2-----"); Animalkeepers leo = new Animalkeepers("Leo"); Animalkeepers theo = new Animalkeepers("Theo"); Animalkeepers mochi = new Animalkeepers("Mochi"); System.out.println("-----3-----"); Animalkeepers.printTasks(); System.out.println("-----4-----"); leo.doTask("Lion", "Feed"); System.out.println("-----5-----"); leo.doTask("Monkey", "Feed"); System.out.println("-----6-----"); Animalkeepers.details(); System.out.println("-----7-----"); theo.doTask("Tiger", "Bathe"); mochi.doTask("Seal", "Clean Pen"); mochi.doTask("Deer", "Add Food"); System.out.println("-----8-----"); Animalkeepers.printTasks(); System.out.println("-----9-----"); leo.doTask("Deer", "Play"); System.out.println("-----10-----"); Animalkeepers.details(); } }</pre>	<pre>No Animal Keepers working yet. -----1----- [Lion, Tiger, Seal, Gorilla, Deer] -----2----- Leo with ID 101 got the job! Theo with ID 102 got the job! Mochi with ID 103 got the job! -----3----- No tasks assigned. -----4----- Task assigned to Leo -----5----- Animal not in the Safari -----6----- Total Animal Keeper: 3 Total Task assigned: 1 Feed (Keeper - Leo) === Lion -----7----- Task assigned to Theo Task assigned to Mochi Task assigned to Mochi -----8----- Feed (Keeper - Leo) === Lion Bathe (Keeper - Theo) === Tiger Clean Pen (Keeper - Mochi) === Seal Add Food (Keeper - Mochi) === Deer -----9----- Task assigned to Leo -----10----- Total Animal Keeper: 3 Total Task assigned: 4 Feed (Keeper - Leo) === Lion Bathe (Keeper - Theo) === Tiger Clean Pen (Keeper - Mochi) === Seal Play (Keeper - Leo) === Deer</pre>

Task 5

Design the **Event** and **Organizer** classes in such a way that the following code provides the expected output. Hint:

- Make the name instance variable of the Event class **private**
- For simplicity assume that the Event class can create a maximum of 5 event objects and an Organizer can organize a maximum of 4 events.

Driver Code	Output
<pre>public class EventTester{ public static void main(String args []){ Event.allEventInfo(); System.out.println("1-----"); Event ev1 = new Event("HP Day", "7/12/24"); Event ev2 = new Event("TechConnect", "10/12/24"); System.out.println(ev1.details()); System.out.println("2-----"); Organizer uni = new Organizer(); Organizer bracu = new Organizer("BRACU"); Organizer buet = new Organizer("BUET"); System.out.println("3-----"); Event.allEventInfo(); System.out.println("4-----"); bracu.organizeEvent(ev1); bracu.organizeEvent(ev2); System.out.println("5-----"); Event ev3 = new Event("From Earth to Orbit", "15/12/24"); Event ev4 = new Event("NSysS 2024","21/12/24"); System.out.println("6-----"); buet.organizeEvent(ev4); bracu.organizeEvent(ev3); System.out.println("7-----"); bracu.searchEventByDate("21/12/24"); System.out.println("8-----"); bracu.searchEventByDate("15/12/24"); System.out.println("9-----"); Event.allEventInfo(); } }</pre>	<pre>Total Events: 0 Event Details: 1----- Name: HP Day Date: 7/12/24 2----- Please provide the organizer's name 3----- Total Events: 2 Event Details: Event 1: Name: HP Day Date: 7/12/24 Event 2: Name: TechConnect Date: 10/12/24 4----- BRACU successfully organized HP Day BRACU successfully organized TechConnect 5----- 6----- BUET successfully organized NSysS 2024 BRACU successfully organized From Earth to Orbit 7----- No event is scheduled for 21/12/24 8----- From Earth to Orbit 9----- Total Events: 4 Event Details: Event 1: Name: HP Day Date: 7/12/24 Event 2: Name: TechConnect Date: 10/12/24 Event 3: Name: From Earth to Orbit Date: 15/12/24 Event 4: Name: NSysS 2024 Date: 21/12/24</pre>

Task 6

1	class Trace {
2	public static int[] x = {3, -4};
3	public int y = 4;
4	public static int temp = -5;
5	private int sum = 2;
6	public Trace(){
7	y = temp + 3 ;
8	sum = 3 + temp + x[1];
9	temp-=2;
10	x[0] = ++x[1] - 2;
11	}
12	public Trace(Trace trace){
13	sum = trace.sum;
14	x = trace.x;
15	trace.methodB(1,3);
16	}
17	public void methodA(int m, int n){
18	int x = 2 - this.x[0] - Trace.x[1];
19	y = y + m + (temp++);
20	x = x + 7 + n;
21	sum = sum + x + y;
22	System.out.println(x + " " + y+ " " + sum);
23	}
24	public void methodB(int m, int n){
25	int y = 0;
26	y = y + this.y;
27	Trace.x[0] = this.y + 3 + temp;
28	methodA(x[1], y);
29	sum = Trace.x[1] + y + sum;
30	System.out.println(this.x[0] + " " + y+ " " + sum);
31	}
32	public static void methodC(Trace trace1, Trace trace2){
33	temp = x[0] - Trace.x[1];
34	x = new int[]{trace1.y, trace2.y};
35	}
36	}

Consider the following driver code and find the output.

Trace trace1 = new Trace();	Output 1	Output 2	Output 3
Trace trace2 = new Trace(trace1);			
trace1.methodA(3, 2);			
Trace.methodC(trace1, trace2);			
trace2.methodB(1, 2);			

Task 7

1	class Tracing {
2	public static int x = 0, y = 0;
3	public int a;
4	private int b = 3;
5	public Tracing(int a, int b) {
6	this.a = a;
7	this.b = b - this.b;
8	x += 1;
9	y += Tracing.y - 2;
10	}
11	public void set_b(int b) {
12	this.b = b;
13	}
14	public int get_b() {
15	return this.b;
16	}
17	public void methodA(int x) {
18	this.a = x + this.x - Tracing.x;
19	this.b = this.a + this.methodB() - this.b;
20	System.out.println(this.a + " " + this.b + " " + x);
21	}
22	public int methodB() {
23	int y = -3;
24	this.b = y - this.y + this.a;
25	System.out.println(this.a + " " + this.b + " " + x);
26	this.y -= y;
27	x += this.b + this.y;
28	return this.b;
29	}
30	public void methodB(Tracing t1) {
31	int t = this.y - t1.get_b() + this.b;
32	t1.set_b(t);
33	t1.a = this.x - t1.a + this.a;
34	System.out.println(t1.a + " " + t1.get_b() + " " + x);
35	}
36	}

Consider the following driver code and find the output.

Tracing t1 = new Tracing(2, 3);	Output 1	Output 2	Output 3
t1.methodA(1);			
Tracing t2 = new Tracing(3, 4);			
t2.methodA(2);			
t1.methodB(t2);			
t2.methodB(t2);			

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Design the **SultansDine** class with the necessary property to produce the output from the given driver code.

Subtasks:

1. Create SultansDine class
2. Create 2 static variable and 1 static array
3. Create 1 static method
4. Calculation of branch sell is given below
 - a. If sellQuantity < 10:
 - i. Branch_sell = quantity * 300
 - b. Else if sellQuantity < 20:
 - i. Branch_sell = quantity * 350
 - c. Else
 - i. Branch_sell = quantity * 400
5. Calculation of branch's sell percentage = (branch's sell / total sell) * 100

Driver Code	Output
<pre>public class SultansDineTester { public static void main(String[] args) { SultansDine.details(); System.out.println("1====="); SultansDine dhanmondi = new SultansDine("Dhanmondi"); dhanmondi.sellQuantity(25); dhanmondi.branchInformation(); System.out.println("2====="); SultansDine.details(); System.out.println("3====="); SultansDine baily_road = new SultansDine("Baily Road"); baily_road.sellQuantity(15); baily_road.branchInformation(); System.out.println("4====="); SultansDine.details(); System.out.println("5====="); SultansDine gulshan = new SultansDine("Gulshan"); gulshan.sellQuantity(9); gulshan.branchInformation(); System.out.println("6====="); SultansDine.details(); } }</pre>	<pre>Total Number of branch(s): 0 Total Sell: 0 Taka 1===== Branch Name: Dhanmondi Branch Sell: 10000 Taka 2===== Total Number of branch(s): 1 Total Sell: 10000 Taka Branch Name: Dhanmondi, Branch Sell: 10000 Taka Branch consists of total sell's 100.00 3===== Branch Name: Baily Road Branch Sell: 5250 Taka 4===== Total Number of branch(s): 2 Total Sell: 15250 Taka Branch Name: Dhanmondi, Branch Sell: 10000 Taka Branch consists of total sell's 65.57 Branch Name: Baily Road, Branch Sell: 5250 Taka Branch consists of total sell's 34.43 5===== Branch Name: Gulshan Branch Sell: 2700 Taka 6===== Total Number of branch(s): 3 Total Sell: 17950 Taka Branch Name: Dhanmondi, Branch Sell: 10000 Taka Branch consists of total sell's 55.71 Branch Name: Baily Road, Branch Sell: 5250 Taka Branch consists of total sell's 29.25 Branch Name: Gulshan, Branch Sell: 2700 Taka Branch consists of total sell's 15.04</pre>

Task 2

Design a **Student** class in such a way that the following code provides the expected output.

Driver Code	Output
<pre>public class StudentTester { public static void main(String[] args) { Student.printDetails(); System.out.println("-----"); Student mikasa = new Student("Mikasa", 3.75); mikasa.individualDetail(); System.out.println("-----"); Student.printDetails(); System.out.println("-----"); Student harry = new Student("Harry", 2.5, "Charms"); harry.individualDetail(); System.out.println("-----"); Student.printDetails(); System.out.println("-----"); Student levi = Student.createStudent("Levi", 3.33); levi.individualDetail(); System.out.println("-----"); Student.printDetails(); } }</pre>	<pre>Total Student(s): 0 CSE Student(s): 0 Other Department Student(s): 0 ----- ID: 1 Name: Mikasa CGPA: 3.75 Department: CSE ----- Total Student(s): 1 CSE Student(s): 1 Other Department Student(s): 0 ----- ID: 2 Name: Harry CGPA: 2.5 Department: Charms ----- Total Student(s): 2 CSE Student(s): 1 Other Department Student(s): 1 ----- ID: 3 Name: Levi CGPA: 3.33 Department: CSE ----- Total Student(s): 3 CSE Student(s): 2 Other Department Student(s): 1</pre>

Task 3

For this task, you need to design the **Cargo** class with appropriate static and non-static variables and methods to produce this given output for the given tester code.

Note: .load() method marks an object as selected for transport, and .unload() method unmarked it. At a time, the transport capacity is 10.0 tonnes. Each Cargo object is initialized with 2 attributes from the constructor - the contents and the weight. Carefully observe the outputs to identify the other attributes and design the class.

Given Code	Expected Output
<pre>public class CargoTester { public static void main(String[] args) { System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("1====="); Cargo a = new Cargo("Industrial Machinery", 4.5); a.details(); System.out.println("2====="); a.load(); System.out.println("3====="); Cargo b = new Cargo("Steel Ingot", 2.7); b.details(); System.out.println("4====="); System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("5====="); b.load(); System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("6====="); Cargo c = new Cargo("Tree Trunks", 3.6); c.load(); System.out.println("7====="); c.details(); b.details(); System.out.println("8====="); Cargo d = new Cargo("Processed Goods", 1.8); d.load(); System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("9====="); b.unload(); System.out.println("Cargo Capacity: "+ Cargo.capacity()); System.out.println("10====="); c.load(); System.out.println("11====="); b.details(); System.out.println("Cargo Capacity: "+ Cargo.capacity()); } }</pre>	<pre>Cargo Capacity: 10.0 1===== Cargo ID: 1, Contents: Industrial Machinery, Weight: 4.5, Loaded: false 2===== Cargo 1 loaded for transport. 3===== Cargo ID: 2, Contents: Steel Ingot, Weight: 2.7, Loaded: false 4===== Cargo Capacity: 5.5 5===== Cargo 2 loaded for transport. Cargo Capacity: 2.8 6===== Cannot load cargo, exceeds weight capacity. 7===== Cargo ID: 3, Contents: Tree Trunks, Weight: 3.6, Loaded: false Cargo ID: 2, Contents: Steel Ingot, Weight: 2.7, Loaded: true 8===== Cargo 4 loaded for transport. Cargo Capacity: 1.0 9===== Cargo 2 unloaded. Cargo Capacity: 3.7 10===== Cargo 3 loaded for transport. 11===== Cargo ID: 2, Contents: Steel Ingot, Weight: 2.7, Loaded: false Cargo Capacity: 0.09999999999999964</pre>

Task 4

1.	public class Maze{	Output	
2.	public static int x;		
3.	public void methodA(){		
4.	int m = 5;		
5.	x=11;		
6.	System.out.println(x+" "+m);		
7.	m=methodB(m-3)+x;		
8.	System.out.println(x+" "+(m));		
9.	methodB(x,m);		
10.	System.out.println(x+" "+m+x);		
11.	}		
12.	public int methodB(int y){		
13.	x=y*y;		
14.	System.out.println(x+" "+y);		
15.	return x+3;		
16.	}		
17.	public void methodB(int z, int x){		
18.	z=z-2;		
19.	x=x*1%z;		
20.	System.out.println(z+" "+x);		
21.	}		
22.	}		
23.	public class TestU3{		
24.	public static void main(String [] args){		
25.	Maze c = new Maze();		
26.	c.methodA();		
27.	c.methodB(-11, 45);		
28.	}		
29.	}		

Task 5

1	class TraceTask {
2	public static int x = 3;
3	public int y = 5;
4	public int z = 2;
5	public TraceTask(int z) {
6	this.z = z;
7	x += 2;
8	this.y = x - this.z;
9	}
10	public void methodA(int x) {
11	this.y = x + this.y + TraceTask.x;
12	x = this.z + 4;
13	TraceTask.x = this.y - x;
14	System.out.println(x + " " + this.y + " " + TraceTask.x);
15	}
16	public int methodB(TraceTask t, int z) {
17	int y = 2;
18	t.z = this.z + y;
19	t.methodA(y);
20	this.x = t.y + this.z;
21	System.out.println(this.z + " " + t.z + " " + y);
22	return this.x;
23	}
24	}

<pre> class TraceTaskTester{ public static void main(String[] args) { TraceTask t1 = new TraceTask(4); TraceTask t2 = new TraceTask(2); t1.methodA(3); int res = t2.methodB(t1, 5); System.out.println(res + " " + TraceTask.x + " " + t1.z); } } </pre>	OUTPUT		

Lab Assignment 06



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Inheritance
Number of Tasks:	12 (Classwork: 06, Homework: 06)

[Submit all the Coding Tasks (Homework: Task 1 to 5) in the Google Form shared on buX before the next lab.]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

Complete the class **Circle** so that the desired outputs are generated properly.

Given Code	Expected Output
<pre>public class shapeTester { public static void main(String[] args) { Shape s = new Shape(); s.name = "Mobius Strip"; s.color = "Blue"; s.displayInfo(); System.out.println("1====="); Circle c = new Circle(); System.out.println("2====="); c.name = "Circle"; c.color = "Red"; c.radius = 5; c.displayInfo(); System.out.println("3====="); c.area(); } } public class Shape { public String name; public String color; public void displayInfo() { System.out.printf("Name: %s\nColor: %s\n", name, color); } } public class Circle extends Shape { //Your Code Here }</pre>	<pre>Name: Mobius Strip Color: Blue 1===== 2===== Name: Circle Color: Red 3===== Area of Red Circle: 78.54</pre>

Task 2

Given the following classes, write the code for the **BBAStudent** class so that the following output is printed when we run the TestStudent class.

Driver Code and Parent Class	Output
<pre>public class TestStudent{ public static void main(String [] args){ BBAStudent b1 = new BBAStudent(); BBAStudent b2 = new BBAStudent("Humty Dumty"); BBAStudent b3 = new BBAStudent("Little Bo Peep"); b1.details(); System.out.println("1-----"); b2.details(); System.out.println("2-----"); b3.details(); } } public class Student{ private String name = "Just a Student"; private String department = "nothing"; public void setDepartment(String dpt){ this.department = dpt; } public void setName(String name){ this.name = name; } public void details(){ System.out.println("Name : " + name + " Department: " + department); } }</pre>	<pre>Name: Default Department: BBA 1----- Name: Humty Dumty Department: BBA 2----- Name: Little Bo Peep Department: BBA</pre>

Task 3

Given the following classes, write the code for the **Vehicle2010** class to print the following output when we run the Vehicle2010User class.

Driver Code and Parent Class	Output
<pre>public class Vehicle2010User{ public static void main(String[] args){ Vehicle2010 car1 = new Vehicle2010(); System.out.println(car1.getPosition()); car1.moveLowerLeft(); System.out.println(car1.getPosition()); Vehicle2010 car2 = new Vehicle2010(); System.out.println(car2.getPosition()); car2.moveUpperRight(); System.out.println(car2.getPosition()); car2.moveLowerRight(); System.out.println(car2.getPosition()); } } public class Vehicle{ public int x; public int y; public void moveUp(){ y = y+1; } public void moveDown(){ y = y-1; } public void moveLeft(){ x = x-1; } public void moveRight(){ x = x+1; } public String getPosition(){ return "("+ x + ","+ y + ")"; } }</pre>	<pre>(0,0) (-1,-1) (0,0) (1,1) (2,0)</pre>

Task 4

Given the following classes, write the code for the **Cricket_Tournament** and the **Tennis_Tournament** classes derived from **Tournament** so that the following output is generated.

Given Code	Expected Output
<pre>public class TournamentTester { public static void main(String[] args) { Cricket_Tournament ct1 = new Cricket_Tournament(); System.out.println(ct1.info()); System.out.println("-----"); Cricket_Tournament ct2 = new Cricket_Tournament("IPL", 10, "t20"); System.out.println(ct2.info()); System.out.println("-----"); Tennis_Tournament tt = new Tennis_Tournament("Roland Garros", 128); System.out.println(tt.info()); } } public class Tournament { private String name; public Tournament() { this.name = "Default"; } public Tournament(String name) { this.name = name; } public String getName(){ return "Tournament Name: "+name; } }</pre>	<pre>Cricket Tournament Name: Default Number of Teams: 0 Type: No type ----- Cricket Tournament Name: IPL Number of Teams: 10 Type: t20 ----- Tennis Tournament Name: Roland Garros Number of Players: 128</pre>

Task 5

Design the **Dog** and **Cat** class derived from the **Animal** class with appropriate attributes and properties so that the driver code can generate the output given below.

Driver Code and Parent Class	Output
<pre>public class AnimalTester1 { public static void main(String[] args) { Dog dog = new Dog("Buddy", 5, "Brown", "Bulldog"); Cat cat = new Cat("Kitty", 3, "White", "Persian"); System.out.println("1.====="); System.out.println(dog.dogInfo()); System.out.println("2.====="); System.out.println(cat.catInfo()); System.out.println("3.====="); dog.makeSound(); System.out.println("4.====="); cat.makeSound(); } } public class Animal { public String name; public int age; public String color; public Animal(String name, int age, String color) { this.name = name; this.age = age; this.color = color; } public String info() { return "Name: "+name+"\nAge: "+age+"\nColor: "+color+"\n"; } }</pre>	<pre>1.===== Name: Buddy Age: 5 Color: Brown Breed: Bulldog 2.===== Name: Kitty Age: 3 Color: White Breed: Persian 3.===== Brown color Buddy is barking 4.===== White color Kitty is meowing</pre>

Task 6

1	public class A{
2	public int temp = 4;
3	public int sum = 1;
4	public int y = 2;
5	public void methodA(int m, int n){
6	int x = 0;
7	y = y + m + (temp++);
8	x = x + 2 + n;
9	sum = sum + x + y;
10	System.out.println(x + " " + y+ " " + sum);
11	}
12	}
13	public class B extends A {
14	public int x = 6;
15	public void methodB(int m, int n){
16	int y = 0;
17	y = ++y + this.y;
18	x = this.y + 2 + temp;
19	methodA(x, y);
20	sum = x + y + super.sum;
21	System.out.println(x + " " + y+ " " + sum);
22	}
23	}

<pre> A a1 = new A(); a1.methodA(1, 1); B b1 = new B(); b1.methodB(1, 2); </pre>	x	y	sum

HOMEWORK

Task 1

Complete the class **Cow** so that the desired outputs are generated properly.

Given Code	Expected Output
<pre>public class AnimalTester2 { public static void main(String args[]){ Animal a1 = new Animal(); System.out.println("1-----"); a1.details(); System.out.println("2-----"); Cow c1 = new Cow(); c1.name = "Pammy"; System.out.println("3-----"); System.out.println("Name: " + c1.getName()); c1.details(); System.out.println("4-----"); c1.updateSound("Moo"); System.out.println("5-----"); c1.details(); } } public class Animal{ public int legs = 4; public String sound = "Not defined"; public void details(){ System.out.println("Legs: "+legs); System.out.println("Sound: "+sound); } } public class Cow extends Animal{ //Your Code Here }</pre>	<pre>1----- Legs: 4 Sound: Not defined 2----- The cow says hello! 3----- Name: Pammy Legs: 4 Sound: Not defined 4----- 5----- Legs: 4 Sound: Moo</pre>

Task 2

Design the **CheckingAccount** class derived from the **Account** class with appropriate attributes and properties so that the driver code can generate the output given below.

Driver Code and Parent Class	Output
<pre>public class Account{ public double balance = 0.0; public Account(double balance){ this.balance = balance; } public double showBalance(){ return balance; } } //Tester Class public class TestAccount{ public static void main(String [] args){ System.out.println("Total Checking Accounts: "+CheckingAccount.count); CheckingAccount c1 = new CheckingAccount(); System.out.println("Account Balance: " + c1.showBalance()); CheckingAccount c2 = new CheckingAccount(100.0); System.out.println("Account Balance: " + c2.showBalance()); CheckingAccount c3 = new CheckingAccount(200.0); System.out.println("Account Balance: " + c3.showBalance()); System.out.println("Total Checking Accounts: "+CheckingAccount.count); } }</pre>	<pre>Total Checking Accounts: 0 Account Balance: 0.0 Account Balance: 100.0 Account Balance: 200.0 Total Checking Accounts: 3</pre>

Task 3

Given the following classes, write the code for the **Book** and the **CD** class so that the following output is printed.

Driver Code and Parent Class	Expected Output
<pre>public class Tester6 { public static void main(String[] args) { Book book = new Book(1, "The Alchemist", 500, "97806", "HarperCollins"); System.out.println(book.printDetail()); System.out.println("-----"); CD cd = new CD(2, "Shotto", 300, "Warfaze", 50, "Hard Rock"); System.out.println(cd.printDetail()); } } class Product { private int id; private String title; private int price; public Product(int id, String title, int price) { this.id = id; this.title = title; this.price = price; } public String getIdTitlePrice() { return "ID: " + id + " Title: " + title + " Price: " + price; } }</pre>	<pre>ID: 1 Title: The Alchemist Price: 500 ISBN: 97806 Publisher: HarperCollins ----- ID: 2 Title: Shotto Price: 300 Band: Warfaze Duration: 50 minutes Genre: Hard Rock</pre>

Task 4

Design the **ComplexNumber** class with the necessary property to produce the output from the given driver code.

Driver Code and Parent Class	Output
<pre>public class ComplexNumberTester { public static void main(String[] args) { ComplexNumber cn1 = new ComplexNumber(); System.out.println(cn1.details()); System.out.println("-----"); ComplexNumber cn2 = new ComplexNumber(5.0, 7.0); System.out.println(cn2.details()); } } public class RealNumber { public double realValue; public RealNumber() { this(0.0); } public RealNumber(double realValue) { this.realValue = realValue; } public String getReal(){ return "RealPart: " + realValue; } }</pre>	<pre>RealPart: 1.0 ImaginaryPart: 1.0 ----- RealPart: 5.0 ImaginaryPart: 7.0</pre>

Task 5

Given the following classes, write the code for the **CSEStudent** class derived from **Student** so that the following output is generated.

Given Code	Expected Output
<pre>public class StudentTester{ public static void main (String args[]){ CSEStudent.details(); System.out.println("1-----"); CSEStudent s1 = new CSEStudent("Bob", 23); s1.info(); System.out.println("2-----"); CSEStudent s2 = new CSEStudent("Don", 33); s2.info(); System.out.println("3-----"); s1.addLabBasedCourse("CSE220"); s1.addLabBasedCourse("CSE221"); System.out.println("4-----"); s1.info(); System.out.println("5-----"); CSEStudent.details(); System.out.println("6-----"); s1.addLabBasedCourse("CSE230"); System.out.println("7-----"); s1.info(); System.out.println("8-----"); s2.addLabBasedCourse("CSE110"); s2.info(); } } class Student{ public String name; public int id; public String courses = ""; public Student(String n, int i){ name = n; id = i; } public void info(){ System.out.println("Name: "+name); System.out.println("ID: "+id); System.out.println("Courses: "+courses); } }</pre>	<pre>Total CSE Students: 0 Available Lab Based Courses: CSE110 CSE111 CSE220 CSE221 1----- Name: Bob ID: 23 Courses: 2----- Name: Don ID: 33 Courses: 3----- 4----- Name: Bob ID: 23 Courses: CSE220 CSE221 5----- Total CSE Students: 2 Available Lab Based Courses: CSE110 CSE111 CSE220 CSE221 6----- It is not a lab based course! 7----- Name: Bob ID: 23 Courses: CSE220 CSE221 8----- Name: Don ID: 33 Courses: CSE110</pre>

Task 6

1	public class A{
2	public int m = 14, n = -1, p = 3;
3	public static int temp = 11;
4	public void methodA(int m, int n){
5	int x = 0;
6	p = this.p + m + (this.temp++);
7	x = x + 2 + n;
8	this.n = n + B.x + p;
9	System.out.println(this.m + " " + p+ " " + this.n);
10	}
11	}
12	public class B extends A {
13	public static int x = 6;
14	public void methodB(int x, int y){
15	y = ++y + this.p + n;
16	super.p = x + 2 + temp;
17	methodA(temp, x);
18	B.x = this.x + x + super.temp;
19	System.out.println(B.x + " " + y+ " " + A.temp);
20	}
21	}

<pre> B b1 = new B(); b1.methodB(3, 5); A a1 = new A(); a1.methodA(12, -9); </pre>	x	y	sum

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Given the following classes, write the code for the **Player** and the **Manager** classes derived from SportsPerson class so that the following output is printed. To calculate the match earnings use the following formula:

1. Player: $(\text{total_goal} * 1000) + (\text{total_match} * 10)$
2. Manager: $\text{match_win} * 1000$

Given Code	Expected Output
<pre>public class PlayerTester { public static void main(String[] args) { Player playerOne = new Player("Al-Nassr", "Ronaldo", "Striker", 25, 32); playerOne.calculateRatio(); playerOne.printDetails(); System.out.println("-----"); Manager managerOne = new Manager("Real Madrid", "Zidane", "Manager", 25); managerOne.printDetails(); } } class SportsPerson { private String team; private String name; public String role; public double earningPerMatch; public SportsPerson(String teamName, String name, String role){ this.team = teamName; this.name = name; this.role = role; this.earningPerMatch = 0; } public String getNameTeam() { return "Name: " + name + ", Team Name: " + team; } }</pre>	<pre>Name: Ronaldo, Team Name: Al-Nassr Team Role: Striker Total Goal: 25, Total Played: 32 Goal Ratio: 0.78 Match Earning: 25320K ----- Name: Zidane, Team Name: Real Madrid Team Role: Manager Total Win: 25 Match Earning: 25000K</pre>

Lab Assignment 07



Inspiring Excellence

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Inheritance
Number of Tasks:	11 (Classwork: 05, Homework: 06)

[Submit all the Coding Tasks (Homework: Task 1 to 5) in the Google Form shared on buX before the next lab.]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

A retail company only sells two types of products and wants to store them in a catalog. You are given the Tester class and Product class. Design the Clothing and Electronics class by observing the given classes and their outputs.

Given Code	Expected Output
<pre>public class ShoppingCartTester{ public static void main(String[] args) { Product p1 = new Product("Notebook", 4.99); Clothing c1 = new Clothing("T-Shirt", 19.99, "M"); Electronics e1 = new Electronics("Laptop", 999.99, 24); p1.displayInfo(); c1.displayInfo(); e1.displayInfo(); System.out.println("Total products created : " + Product.totalProducts); Electronics e2 = new Electronics("Smartphone", 799.99, 12); e2.displayInfo(false); } } public class Product { public String productName; public double price; public static int totalProducts = 0; public Product(String productName, double price) { this.productName = productName; this.price = price; totalProducts++; } public void displayInfo() { System.out.println("Product: " + productName + ", Price: \$" + price); } }</pre>	<pre>Product: Notebook, Price: \$4.99 Clothing: T-Shirt, Price: \$19.99, Size: M Electronics: Laptop, Price: \$999.99, Warranty: 24 months Total products created: 3 Electronics: Smartphone, Warranty: 12 months</pre>

Task 2

Given the following base class and driver code, write the code for the Warrior and Mage classes derived from the GameCharacter class so that the following output is printed.

Damage Calculation Formulas:

Warrior: (level * strength) + (armor * 50)

Mage: (level * intelligence) + (mana * 10)

Driver Code and Parent Class	Output
<pre>public class CharacterTester { public static void main(String[] args) { Warrior warriorOne = new Warrior("Conan", 100, 150, 15, "Two-Handed Sword"); System.out.println("-----1-----"); warriorOne.calculateDamage(); System.out.println(warriorOne); System.out.println("-----2-----"); Mage mageOne = new Mage("Merlin", 80, 200, 18, "Arcane Staff"); mageOne.calculateDamage(); System.out.println(mageOne); System.out.println("-----3-----"); GameCharacter.printAllCharacters(); } } class GameCharacter { public static String [] allCharacters= new String [5]; public static int characterCount=0; private String characterName; private int level; public String weaponType; public double totalDamage; public GameCharacter(String name, int level, String weapon) { this.characterName = name; this.level = level; this.weaponType= weapon; } public String getCharacterInfo() { return "Name: " + characterName + ", Level: " + level; } public int getLevel(){ return level; } public static void printAllCharacters(){ if (characterCount>0){ System.out.println("Available Characters:"); for(int i=0;i<characterCount;i++){ System.out.println(allCharacters[i]); } } else{ System.out.println("No available character"); } } }</pre>	<pre>A new warrior has arrived -----1----- Name: Conan, Level: 100 Character Type: Warrior Weapon: Two-Handed Sword Strength: 150, Armor: 15 Total Damage: 15750.0 -----2----- A new mage came to live Name: Merlin, Level: 80 Character Type: Mage Weapon: Arcane Staff Intelligence: 200, Mana: 18 Total Damage: 16180.0 -----3----- Available Characters: Conan Merlin</pre>

Task 3

Design the PremiumRide derived from Ride class to generate the given output.

fare += (fare * 20) / 100.0

An extra 20% charge is applied to the total fare if surge is active(true).

If calculateFare(int amount) is called, the fare is discounted with the amount.

Driver Code and Parent Class	Output
<pre>public class RideTester { public static void main(String[] args) { Ride r1 = new Ride(7); PremiumRide r2 = new PremiumRide(15, true); PremiumRide r3 = new PremiumRide(12, false); System.out.println("1====="); System.out.println(r1); System.out.println(r2); System.out.println(r3); System.out.println("2====="); System.out.println("Regular Ride: "+r1.calculateFare()+" TK"); System.out.println("Premium Ride "+r2.getId()+": +r2.calculateFare()+" TK"); System.out.println("Premium Ride "+r3.getId()+": +r3.calculateFare(40)+" TK"); System.out.println("3====="); System.out.println(r3); } } public class Ride { private int distance; public Ride(int distance) { this.distance = distance; } public double calculateFare() { return distance * 10; } public double getDistance() { return distance; } public String toString(){ return "Distance: "+this.distance+" km"; } }</pre>	<pre>1===== Distance: 7 km Distance: 15 km ID: 1-15 Service Charge: 50 TK Surge: true Discount: false Distance: 12 km ID: 2-12 Service Charge: 50 TK Surge: false Discount: false 2===== Regular Ride: 70.0 TK Premium Ride 1-15: 240.0 TK Premium Ride 2-12: 130.0 TK 3===== Distance: 12 km ID: 2-12 Service Charge: 50 TK Surge: false Discount: true</pre>

Task 4

Design the **CinemexTicket** class derived from the **MovieTicket** Class so that the given output is produced:

- ❖ The `seatTypes` and `seatPrices` arrays contain the type of the seat and its corresponding price
- ❖ Night show charge (15% of ticket price) will be applicable if the time is between 6:00 PM - 11:00 PM
- ❖ Unique id for a ticket is generated by:
`MovieName-FirstLetterOfSeatType-TicketCount`
- ❖ You may need to use `.split()` and `Integer.parseInt()` built-in methods

Parent Class

```
public class MovieTicket {
    public static String [] seatTypes = {"Regular", "Premium", "IMAX 3D"};
    public static double [] seatPrices = {300.0, 450.0, 600.0};
    public static int nightShowCharge = 15;
    private String movie;
    public String showtime;
    public String date;
    private double price;
    public String seat;

    public MovieTicket (String movie, String date, String showtime, double price) {
        this.movie = movie;
        this.showtime = showtime;
        this.date = date;
        this.price = price;
        this.seat = "Not Selected";
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public String getMovie() {
        return movie;
    }

    public String toString() {
        return "Movie: " + movie + "\nShowtime: " + showtime + "\nDate: " + date;
    }
}
```

Driver Code	Output
<pre> public class Tester { public static void main(String[] args) { CinemexTicket ticket1 = new CinemexTicket("Deadpool and Wolverine", "18:30", "Action-Comedy", "July 24, 2024"); System.out.println("Total movie ticket(s): " + CinemexTicket.getTotalTickets()); System.out.println("1====="); ticket1.calculateTicketPrice(); System.out.println("2====="); System.out.println(ticket1); System.out.println("3====="); System.out.println(ticket1.confirmPayment()); System.out.println("4====="); System.out.println(ticket1); System.out.println("5====="); CinemexTicket ticket2 = new CinemexTicket("Twisters", "10:00", "Sci-Fi", "August 10, 2024", "Premium"); System.out.println("Total movie ticket(s): " + CinemexTicket.getTotalTickets()); System.out.println("6====="); ticket2.calculateTicketPrice(); System.out.println("7====="); System.out.println(ticket2.confirmPayment()); System.out.println("8====="); System.out.println(ticket2); System.out.println("9====="); System.out.println(ticket2.confirmPayment()); } } </pre>	<pre> Total movie ticket(s): 1 1===== Ticket price is calculated successfully. 2===== Ticket ID: Deadpool and Wolverine-R-1 Movie: Deadpool and Wolverine Showtime: 18:30 Date: July 24, 2024 Genre: Action-Comedy Seat Type: Regular Price(tk): 345.0 Status: Not Paid 3===== Payment Successful. 4===== Ticket ID: Deadpool and Wolverine-R-1 Movie: Deadpool and Wolverine Showtime: 18:30 Date: July 24, 2024 Genre: Action-Comedy Seat Type: Regular Price(tk): 345.0 Status: Paid 5===== Total movie ticket(s): 2 6===== Ticket price is calculated successfully. 7===== Payment Successful. 8===== Ticket ID: Twisters-P-2 Movie: Twisters Showtime: 10:00 Date: August 10, 2024 Genre: Sci-Fi Seat Type: Premium Price(tk): 450.0 Status: Paid 9===== Ticket price is already paid! </pre>

Task 5

1	public class A {
2	public static int temp = 4;
3	public static int x = -10;
4	public int sum = 0;
5	public int y = 0;
7	public A() {
8	y = temp - 2;
9	sum = temp + 1;
10	temp -= 2;
11	}
13	public void methodA(int m, int n) {
14	int x = 0;
15	y = y + m + (temp++);
16	x = x + 1 + n;
17	sum = sum + x + y;
18	System.out.println(x + " " + y + " " + sum);
19	}
20	}
22	public class B extends A {
23	public static int x = 0;
24	public int sum = -6;
25	public B() {
26	sum = 0;
27	y = temp + 3;
28	super.sum = 3 + temp + 2;
29	temp -= 2;
30	}
31	public B(B b) {
33	sum = b.sum + super.sum;
34	x = b.x + 1;
35	b.methodB(2, 3);
36	}
37	public void methodB(int m, int n) {
38	int y = 0;
39	y = y + this.y;
40	x = y + 2 + (++temp);
41	methodA(x, y);
42	sum = x + y + sum;
43	System.out.println(x + " " + y + " " + sum);
44	}
45	}

Write the output of the following code:

<pre>public class Tester { public static void main(String[] args) { A a1 = new A(); B b1 = new B(); B b2 = new B(b1); b1.methodA(2, 3); b2.methodB(3, 8); } }</pre>	Output:		
	x	y	sum

HOMEWORK

Task 1

Implement the design of the Butterfly class and the Moth class so that these classes extend from the Caterpillar class and generate the output as follows.

- Butterfly loses 5 units of energy due to flying and gains 5 energy units per food amount
- Moth loses 2.5 units of energy due to nocturnal activity and gains 2.5 energy units per food amount

Driver Code and Given Class	Expected Output
<pre>public class Caterpillar_Tester { public static void main(String[] args) { Caterpillar c = new Caterpillar("Leaf", 5); c.showDetails(); System.out.println("-----1-----"); c.eat(); c.eat(3); c.showDetails(); System.out.println("-----2-----"); Butterfly b = new Butterfly("Flower", 2); Moth m = new Moth("Cotton", 4); System.out.println("-----3-----"); b.transform(); m.transform(); System.out.println("-----4-----"); b.eat(2); m.eat(5); System.out.println("-----5-----"); b.showDetails(); m.showDetails(); } } public class Caterpillar { public String food; public int age; public double energy; public Caterpillar(String food, int age) { this.food = food; this.age = age; this.energy = 0; } public void eat() { energy += 1; System.out.println("Caterpillar ate 1 unit of food"); } public void eat(int amount) { energy += amount; System.out.println("Caterpillar ate " + amount + " units of food"); } }</pre>	<pre>Food: Leaf Age: 5 Energy: 0.0 -----1----- Caterpillar ate 1 unit of food Caterpillar ate 3 units of food Food: Leaf Age: 5 Energy: 4.0 -----2----- -----3----- Caterpillar transforms into Butterfly Caterpillar transforms into Moth -----4----- Butterfly lost energy while flying and absorbed nectar Moth lost energy due to nocturnal activity -----5----- Food: Flower Age: 2 Energy: 5.0 joules Food: Cotton Age: 4 Energy: 10.0 joules</pre>

<pre> public void transform() { System.out.println("Caterpillar transforms"); } public void showDetails() { System.out.println("Food: " + food); System.out.println("Age: " + age); System.out.println("Energy: " + energy); } } </pre>	
---	--

Task 2

You are given the Audience class. You need to design the Concert (Parent) and VIPConcert (Child) classes so that the following tester code generates the desired output.

Hint: Assume a maximum of 5 artists can be added to a concert. Ticket Pricing:

- Zone A: 500 per ticket
- Zone B: 1,000 per ticket
- Zone VIP: 2,000 per ticket

Driver Code and Given Class	Output
<pre> public class ConcertTester{ public static void main(String[] args) { Concert concert = new Concert("ABC Conference Center","7:00 PM"); concert.showDetails(); System.out.println("-----1-----"); concert.addArtist("Tahsan Khan"); concert.addArtist("Habib Wahid"); concert.showDetails(); System.out.println("-----2-----"); Audience a1 = new Audience("Sarah"); Audience a2 = new Audience("Alex"); System.out.println("-----3-----"); a1.buyConcertTicket(concert, "A", 3); a2.buyConcertTicket(concert, "B"); System.out.println("-----4-----"); concert.showDetails(); System.out.println("-----5-----"); VIPConcert vipConcert = new VIPConcert("Army Stadium","10:00 PM"); vipConcert.addArtist("Atif Aslam"); Audience a3 = new Audience("Emily"); a3.buyConcertTicket(vipConcert); System.out.println("-----6-----"); vipConcert.showDetails(); System.out.println("-----7-----"); Concert.showTotalTicketsSold(); } } public class Audience { private String name; </pre>	<pre> Venue: ABC Conference Center Showtime: 7:00 PM Artist: Tickets sold in this concert: 0 -----1----- Venue: ABC Conference Center Showtime: 7:00 PM Artist: -Tahsan Khan -Habib Wahid Tickets sold in this concert: 0 -----2----- -----3----- Sarah bought 3 ticket(s) in Zone A Total price: 1500 Alex bought 1 ticket(s) in Zone B Total price: 1000 -----4----- Venue: ABC Conference Center Showtime: 7:00 PM Artist: -Tahsan Khan -Habib Wahid Tickets sold in this concert: 4 -----5----- Emily bought 1 ticket(s) in Zone VIP Total price: 2000 -----6----- Venue: Army Stadium Showtime: 10:00 PM </pre>

```
public Audience(String name) {
    this.name = name;
}
public void buyConcertTicket(Concert concert, String zone, int
quantity) {
    System.out.println(name + " bought " + quantity + " ticket(s) in
Zone " + zone);
    concert.buyTicket(zone, quantity);
}
public void buyConcertTicket(Concert concert, String zone) {
    System.out.println(name + " bought 1 ticket(s) in Zone " + zone);
    concert.buyTicket(zone);
}
public void buyConcertTicket(VIPConcert concert) {
    System.out.println(name + " bought 1 ticket(s) in Zone VIP");
    concert.buyTicket();
}
}
```

Artist:
-Atif Aslam
Tickets sold in this concert: 1
-----7-----
Total tickets sold (all venues): 5

Task 3

Given the following classes, write the code for the **CarDriver** and the **BikeDriver** class so that the following output is printed.

Driver Code and Parent Class	Expected Output
<pre> import java.util.Arrays; public class PathaoRideManager{ public static void main(String[] args) { System.out.println("1.====="); GenericDriver d = new GenericDriver(); System.out.println("2.====="); d.hasSafetyTraining(); System.out.println("3.====="); BikeDriver d1 = new BikeDriver("John", "Not Premium"); System.out.println("4.====="); System.out.println(d1); System.out.println("5.====="); System.out.println(d1.acceptRide(false)); System.out.println("6.====="); d1.hasSafetyTraining(); System.out.println("7.====="); CarDriver d2 = new CarDriver("Max"); System.out.println("8.====="); System.out.println(d2); System.out.println("9.====="); d2.hasSafetyTraining(); System.out.println("10.====="); System.out.println(d2.acceptRide(true)); System.out.println("11.====="); System.out.println(d2); System.out.println("12.====="); BikeDriver.restrictedAreas("Airport Road"); BikeDriver.restrictedAreas("Defense Area"); BikeDriver.restrictedAreas("Navy Base"); System.out.println(Arrays.toString(BikeDriver.restrictedAreas)); System.out.println("13.====="); d1.fightRestriction(new String[]{"Defense Area", "Banani", "Uttara"}); System.out.println("14.====="); CarDriver.restrictedAreas("Cantonment"); CarDriver.restrictedAreas("Road 27"); System.out.println(Arrays.toString(CarDriver.restrictedAreas)); System.out.println("15.====="); d2.fightRestriction(new String[]{"Defense Area", "Road 27"}); } } public class GenericDriver { private String name; public GenericDriver() { System.out.println("Welcome to Pathao!"); } public GenericDriver(String name) { this.name = name; System.out.println("Welcome to Pathao Driver Panel!"); } } </pre>	<pre> 1.===== Welcome to Pathao! 2.===== All drivers must have safety training. 3.===== Welcome to Pathao Driver Panel! John has been registered as a Pathao driver! 4.===== John's driver profile is Not Premium 5.===== All Pathao drivers can accept rides. Driver does not have a verified vehicle. John's driver profile is Not Premium 6.===== All drivers must have safety training. 7.===== Welcome to Pathao Driver Panel! Max has been registered as a Pathao driver! 8.===== Max's driver profile is a Premium 9.===== All drivers must have safety training. Premium drivers receive extra safety briefings. 10.===== All Pathao drivers can accept rides. Driver has a verified vehicle. Max's driver profile is Premium 11.===== Max's driver profile is a Premium 12.===== [Airport Road, Defense Area, Navy Base] 13.===== John cannot enter Defense Area John can enter Banani John can enter Uttara 14.===== [Cantonment, Road 27] 15.===== Max can enter Defense Area Max cannot enter Road 27 </pre>

```

    System.out.println(this.name + " has been registered as a
Pathao driver!");
}
public String getName() {
    return name;
}
public void hasSafetyTraining() {
    System.out.println("All drivers must have safety training.");
}
public String acceptRide(boolean hasVehicle) {
    String s = "All Pathao drivers can accept rides. ";
    if (!hasVehicle)
        s += "Driver does not have a verified vehicle. ";
    else
        s += "Driver has a verified vehicle. ";
    return s;
}
}

```

Task 4

Design the Car and ElectricCar classes so that the following output is produced. The ElectricCar class and Car class should inherit from the Vehicle class.

Driver Code and Parent Class	Output
<pre> public class VehicleShowroom { public static void main(String[] args) { Car c1 = new Car("Toyota Camry", 25000, 4); System.out.println("-----1-----"); c1.vehicleDetail(); System.out.println("-----2-----"); Car.showAllAvailableCars(); System.out.println("-----3-----"); Car c2 = new Car("Honda Civic", 22000, 4); Car c3 = new Car("Ford Mustang", 35000, 2); Car.markAsSold(c1); Car.markAsSold(c2); System.out.println("-----4-----"); c2.vehicleDetail(); System.out.println("-----5-----"); Car.showAllCars(); System.out.println("-----6-----"); ElectricCar e1 = new ElectricCar("Tesla Model 3", 45000, 75); ElectricCar e2 = new ElectricCar("Nissan Leaf", 32000, 60); ElectricCar e3 = new ElectricCar("Nissan Leaf", 32000, 60); System.out.println("-----7-----"); e1.vehicleDetail(); System.out.println("-----8-----"); e3.vehicleDetail(); System.out.println("-----9-----"); ElectricCar.markAsSoldEV(e1); ElectricCar.markAsSoldEV(e2); ElectricCar.markAsSoldEV(e3); } } </pre>	<pre> Vehicle ID: CAR001 created -----1----- Model: Toyota Camry, Price: \$25000 Status: Available Type: Regular Car Seats: 4 -----2----- Total Car: 1 Available Cars: CAR001 : Toyota Camry -----3----- Vehicle ID: CAR002 created Vehicle ID: CAR003 created -----4----- Model: Honda Civic, Price: \$22000 Status: Sold Type: Regular Car Seats: 4 -----5----- Total Car: 3 CAR001 : Toyota Camry - sold CAR002 : Honda Civic - sold CAR003 : Ford Mustang - available -----6----- Vehicle ID: EV001 created Vehicle ID: EV002 created Vehicle ID: EV003 created -----7----- Model: Tesla Model 3, Price: \$45000 </pre>

<pre> public class Vehicle { public String model; public int price; public boolean sold; public String vehicleId; public Vehicle(String model, int price) { this.model = model; this.price = price; this.sold = false; this.vehicleId = ""; } public void vehicleDetail() { System.out.println("Model: " + model + ", Price: \$" + price); System.out.print("Status: "); if(sold){ System.out.print("Sold\n"); } else{ System.out.print("Available\n"); } } } </pre>	<pre> Status: Available Type: Electric Vehicle Battery Capacity: 75 kWh -----8----- Model: Nissan Leaf, Price: \$32000 Status: Available Type: Electric Vehicle Battery Capacity: 60 kWh -----9----- </pre>
--	---

Task 5

Design the **Manager** and **Developer** class derived from the **Employee** class with appropriate attributes and properties so that the driver code can generate the output given below. [Hint:

Manager:

1. Adds a bonus to the base salary if the manager works more than 40 hours.
2. If the manager works more than 100 hours, the full amount is approved; if they work more than 80 hours, half the amount is approved. Otherwise, the increment is denied.

Developer:

1. Adds \$700 to the base salary if the developer works with Java programming language.]

Driver Code and Parent Class	Output
<pre> public class Employee { public String name; private double baseSalary; private int hoursWorked; public Employee(String name, double baseSalary, int hoursWorked){ this.name = name; this.baseSalary = baseSalary; this.hoursWorked = hoursWorked; } public double getBaseSalary() { return baseSalary; } } </pre>	<pre> 1.===== Name: Neymar Base Salary: \$1000.0 Work Hours: 45 Bonus: 10.0 % Final Salary: \$1100.0 2.===== Increment denied. 3.===== \$50 Increment approved. 4.===== 5.===== Name: Neymar </pre>

```

    public void setBaseSalary(double baseSalary) {
        this.baseSalary = baseSalary;
    }
    public int getHoursWorked() {
        return hoursWorked;
    }
    public void setHoursWorked(int hoursWorked) {
        this.hoursWorked = hoursWorked;
    }
    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Base Salary: $" + baseSalary);
        System.out.println("Work Hours: " + hoursWorked);
    }
}

public class EmployeeTester {
    public static void main(String[] args) {
        Manager neymar = new Manager("Neymar", 1000, 45, 10);
        Developer messi = new Developer("Messi", 1000, 50, "Java");
        Developer chiesa = new Developer("Chiesa", 1000, 50, "Javascript");
        neymar.calculateSalary();
        System.out.println("1.=====");
        neymar.displayInfo();
        System.out.println("2.=====");
        neymar.requestIncrement(100);
        System.out.println("3.=====");
        neymar.setHoursWorked(85);
        neymar.requestIncrement(100);
        System.out.println("4.=====");
        neymar.calculateSalary();
        System.out.println("5.=====");
        neymar.displayInfo();
        System.out.println("6.=====");
        messi.calculateSalary();
        System.out.println("7.=====");
        messi.displayInfo();
        System.out.println("8.=====");
        chiesa.calculateSalary();
        System.out.println("9.=====");
        chiesa.displayInfo();
    }
}

```

```

Base Salary: $1050.0
Work Hours: 85
Bonus: 10.0 %
Final Salary: $1155.0
6.=====
7.=====
Name: Messi
Base Salary: $1000.0
Work Hours: 50
Language: Java
Final Salary: $1700.0
8.=====
9.=====
Name: Chiesa
Base Salary: $1000.0
Work Hours: 50
Language: Javascript
Final Salary: $1000.0

```


Task 6

1	public class A {
2	public static int temp = 4;
3	public static int x = -10;
4	public int sum, y;
5	public A() {
6	y = temp - 2;
7	sum = temp + 1 + this.x;
8	temp -= 2;
9	}
10	public A(int x){
11	this.methodA(3,5);
12	}
13	public void methodA(int m, int n) {
14	y = y + m + (temp++);
15	x = x + 1 + n;
16	sum = sum + this.x + y;
17	System.out.println(x + " " + y + " " + sum);
18	}
19	}
20	public class B extends A {
21	public static int x = 0;
22	public int sum = -6;
23	public B() {
24	sum = 0;
25	y = temp + 3;
26	super.sum = this.x + super.x + A.x;
27	B.x -= 2;
28	}
29	public B(B b) {
30	super(5);
31	sum = b.sum + super.sum;
32	x = b.x + 1;
33	b.methodB(2, 3);
34	}
35	public void methodA(int m, int n) {
36	y = this.y + n + sum;
37	x = x + 4 + n;
38	sum = super.sum + x + y;
39	System.out.println(x + " " + y + " " + sum);
40	}
41	public void methodB(int m, int n) {
42	x = y + 2 + (++temp);
43	sum = x + y + sum;
44	System.out.println(x + " " + y + " " + sum);

45	<code>super.methodA(x, y);</code>
46	<code>}</code>
47	<code>}</code>

<code>B b1 = new B(); B b2 = new B(b1); b1.methodA(2, 3);</code>	x	y	sum

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Design the **KKTea** (parent) and **KKFlavouredTea** (child) classes so that the following output is produced. The **KKFlavouredTea** class should inherit **KKTea** and **KKTea** should inherit the **Tea** class. Note that:

- An object of either class represents a single box of teabags.
- Each tea bag weighs 2 grams.
- The status of an object refers to whether it is sold or not

Driver Code and Parent Class	Output
<pre>public class Tea { public String name; protected int price; protected boolean status; public Tea(String name, int price) { this.name = name; this.price = price; this.status = false; } public void productDetail() { System.out.println("Name: " + name + ", Price: " + price); System.out.println("Status: " + status); } } //Driver Code public class TeaTester{ public static void main(String[] args) { KKTea t1 = new KKTea(250, 50); System.out.println("-----1-----"); t1.productDetail(); System.out.println("-----2-----"); KKTea.totalSales(); System.out.println("-----3-----"); KKTea t2 = new KKTea(470, 100); KKTea t3 = new KKTea(360, 75); KKTea.updateSoldStatusRegular(t1); KKTea.updateSoldStatusRegular(t2); System.out.println("-----4-----"); t2.productDetail(); System.out.println("-----5-----"); KKTea.totalSales(); System.out.println("-----6-----"); KKFlavouredTea t4 = new KKFlavouredTea("Jasmine", 260, 50); KKFlavouredTea t5 = new KKFlavouredTea("Honey Lemon", 270, 45); KKFlavouredTea t6 = new KKFlavouredTea("Honey Lemon", 270, 45); System.out.println("-----7-----"); t4.productDetail(); System.out.println("-----8-----"); t6.productDetail(); System.out.println("-----9-----"); KKFlavouredTea.updateSoldStatusFlavoured(t4); KKFlavouredTea.updateSoldStatusFlavoured(t5); KKFlavouredTea.updateSoldStatusFlavoured(t6);</pre>	<pre>-----1----- Name: KK Regular Tea, Price: 250 Status: false Weight: 100, Tea Bags: 50 -----2----- Total Sales: 0 KK Regular Tea: 0 -----3----- -----4----- Name: KK Regular Tea, Price: 470 Status: true Weight: 200, Tea Bags: 100 -----5----- Total Sales: 2 KK Regular Tea: 2 -----6----- -----7----- Name: KK Jasmine Tea, Price: 260 Status: false Weight: 100, Tea Bags: 50 -----8----- Name: KK Honey Lemon Tea, Price: 270 Status: false Weight: 90, Tea Bags: 45 -----9----- -----10----- Total Sales: 5 KK Regular Tea: 2 KK Flavoured Tea: 3</pre>

<pre> System.out.println("-----10-----"); KKTea.totalSales(); } } </pre>	
--	--

Task 2

1	public class A{
2	public static int temp = 4;
3	public int sum = 1;
4	public int y = 2;
5	public A(){
6	y = temp - 2;
7	sum = temp + 3;
8	temp-=2;
9	}
10	public void methodA(int m, int n){
11	int x = 0;
12	y = y + m + (temp++);
13	x = x + 2 + n;
14	sum = sum + x + y;
15	System.out.println(x + " " + y+ " " + sum);
16	}
17	}
18	public class B extends A {
19	public int x = 5, temp = 10;
20	public B(){
21	y = temp + 3 ;
22	sum = 3 + temp + 2;
23	temp-=1;
24	}
25	public B(B b){
26	sum = b.sum;
27	x = b.x;
28	}
29	public void methodB(int m, int n){
30	int y = 0;
31	y = y + this.y;
32	x = this.y + 2 + temp;
33	methodA(x, y);
34	sum = x + y + super.sum;
35	System.out.println(x + " " + y+ " " + sum);
36	}
37	}

A a1 = new A();	x	y	sum
B b1 = new B();			
B b2 = new B(b1);			
a1.methodA(1, 1);			

<code>b1.methodA(1, 2);</code> <code>b2.methodB(3, 2);</code>			

Lab Assignment 08

[HOMEWORK SUBMISSION DEADLINE: 10 MAY, 11:59 PM]



I n s p i r i n g E x c e l l e n c e

Course Code:	CSE111
Course Title:	Programming Language II
Topic:	Polymorphism
Number of Tasks:	8 (Classwork: 05, Homework: 03)

[Submit all the Coding Tasks (Homework: Task 1 to 2) in the Google Form shared on buX before the deadline.]

[You are not allowed to change the given codes of any of the tasks]

CLASSWORK

Task 1

1.a. Write the **Pokemon** class so that the following code generates the output below:

Driver Code	Output
<pre>public class PokemonTester{ public static void main(String[] args){ Pokemon pikachu = new Pokemon("Pikachu"); pikachu.attack(); pikachu.attack("Thunderbolt"); pikachu.attack("Iron Tail", 90); } }</pre>	<pre>Pikachu attacks with a basic move! Pikachu uses Thunderbolt! Pokemon uses Iron Tail with power 90!</pre>

1.b. What type of polymorphism is depicted in the code above?

Task 2

Write the **Mango** and the **Jackfruit** classes derived from **Fruit** class so that the following code generates the output below:

Parent Class	
<pre>public class Fruit{ private boolean formalin = false; private String name = ""; public Fruit(boolean formalin, String name){ this.formalin = formalin; this.name = name; } public String getName(){ return name; } public boolean hasFormalin(){ return formalin; } }</pre>	
Driver Code	Output
<pre>public class FruitTester{ public static void testFruit(Fruit f){ System.out.println("----Printing Detail-----"); if(f.hasFormalin()){ System.out.println("Do not eat the "+f.getName()+"."); System.out.println(f); }else{ System.out.println("Eat the "+f.getName()+"."); System.out.println(f); } } public static void main(String [] args){ Mango m = new Mango(); testFruit(m); Jackfruit j = new Jackfruit(); testFruit(j); } }</pre>	<pre>----Printing Detail----- Do not eat the Mango. Mangos are bad for you ----Printing Detail----- Eat the Jackfruit. Jackfruits are good for you</pre>

Task 3

Design a set of classes for a Fantasy Game Character System. There is a parent class called **Character**. From it, there are two different child classes: **Warrior** and **Mage**. Additionally, there is a subclass called **Paladin** that extends **Warrior**.

Each character has:

- name (String)
- level (int)

Lastly, you need to Override the `.equals()` method inside the parent class

Parent Class	
<pre>public class Character { public String name; public int level; public Character(String name, int level) { this.name = name; this.level = level; } public void specialMove() { System.out.println("Character uses a generic move."); } // Override the .equals() method }</pre>	
Driver Code	Output
<pre>public class GameTester { public static void main(String[] args) { Character c1 = new Paladin("Arthur", 10); Character c2 = new Mage("Merlin", 12); Character c3 = new Warrior("Leon", 10); c1.specialMove(); c2.specialMove(); c3.specialMove(); + if (c1 instanceof Paladin) { Paladin p = (Paladin) c1; p.specialMove(); } Warrior w1 = new Warrior("Leon", 10); System.out.println("c3 equals w1? " + c3.equals(w1)); Mage m1 = new Mage("Merlin", 15); System.out.println("c2 equals m1? " + c2.equals(m1)); } }</pre>	<pre>Arthur unleashes a holy strike! Merlin casts a powerful fireball! Leon performs a heavy sword slash! Arthur unleashes a holy strike! c3 equals w1? true c2 equals m1? false</pre>

Task 4

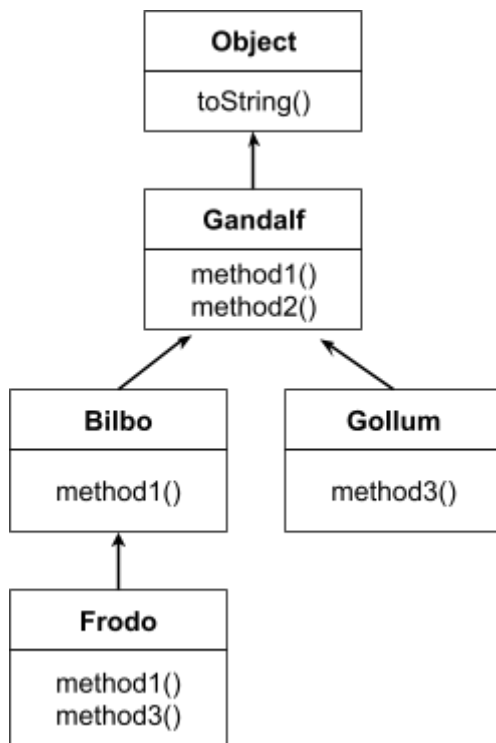
1	public class Gandalf {
2	public void method1(){
3	System.out.println("Gandalf 1");
4	}
5	
6	public void method2(){
7	System.out.println("Gandalf 2");
8	method1();
9	}
10	}
11	public class Bilbo extends Gandalf{
12	public void method1(){
13	System.out.println("Bilbo 1");
14	}
15	}
16	public class Gollum extends Gandalf{
17	public void method3(){
18	System.out.println("Gollum 3");
19	}
20	}
21	public class Frodo extends Bilbo{
22	public void method1(){
23	System.out.println("Frodo 1");
24	super.method1();
25	}
26	
27	public void method3(){
28	System.out.println("Frodo 3");
29	}
30	}

Assuming the following variables have been defined:

```
Gandalf var1 = new Frodo();
Gandalf var2 = new Bilbo();
Gandalf var3 = new Gandalf();
Object var4 = new Bilbo();
Bilbo var5 = new Frodo();
Object var6 = new Gollum();
```

A class diagram for the above classes has been added with this question.

Note: The diagram may not always be provided, so students should know how to draw it from the code.



In the table below,

- The output produced by the statement in the left-hand column, should be written in the right-hand column
- If the statement produces more than one line of output, indicate the line breaks with slashes as in "a/b/c" to indicate three lines of output with "a" followed by "b" followed by "c".
- If the statement causes an error, fill in the right-hand column with either the phrase "compiler error" or "runtime error" to indicate when the error would be Detected.

	Statement	Output
1	var1.method1();	
2	var2.method1();	
3	var4.method1();	
4	var6.method1();	
5	var1.method2();	
6	var3.method2();	
7	var4.method2();	
8	var5.method2();	
9	var6.method2();	

10	((Frodo)var4).method3();	
11	((Frodo)var6).method2();	
12	((Gollum)var1).method3();	
13	((Gollum)var4).method1();	
14	((Gandalf)var1).method2();	
15	((Frodo)var4).method1();	
16	((Gollum)var6).method2();	
17	((Gandalf)var2).method1();	
18	((Bilbo)var6).method2();	
19	((Frodo)var1).method3();	
20	((Gandalf)var5).method3();	

Task 5

1	public class Caramel extends SilkOreo{
2	String texture = "Softy";
3	public void method1() {
4	System.out.println("Caramel m1");
5	}
6	public void method4() {
7	System.out.println("Caramel m4");
8	}
9	public String toString(){
10	method2();
11	return "Caramel is "+ texture;
12	}
13	}
14	public class Chocolate{
15	String texture = "Chocolaty";
16	public void method1() {
17	method2();
18	System.out.println("Chocolate m1");
19	}
20	public void method2() {
21	System.out.println("Chocolate m2");
22	}
23	public String toString(){
24	method2();
25	return "Chocolate is "+ texture;
26	}
27	}
28	public class DairyMilk extends Chocolate{
29	String texture = "Yummy";
30	public void method2() {
31	System.out.println(this.texture);
32	System.out.println("DairyMilk m2");
33	}
34	public void method3() {
35	System.out.println("DairyMilk m3");
36	}
37	}
38	public class KitKat extends Chocolate{
39	String texture = "Crunchy";
40	public void method1() {
41	System.out.println("KitKat m1");
42	}
43	public void method4() {
44	System.out.println("KitKat m4");
45	}

46	public String toString(){
47	method2();
48	return "KitKat is "+ texture;
49	}
50	}
51	public class SilkOreo extends DairyMilk{
52	String texture = "Silky";
53	public void method1() {
54	super.method1();
55	System.out.println("SilkOreo m1");
56	}
57	public void method3() {
58	System.out.println("SilkOreo m3");
59	System.out.println(this);
60	}
61	}

Assuming the following variables have been defined:

```
Chocolate choco1 = new Chocolate();
KitKat kit = new KitKat();
DairyMilk dairyMilk1 = new DairyMilk();
DairyMilk dairyMilk2 = new SilkOreo();
Object obj1 = new DairyMilk();
Object obj2 = new KitKat();
Chocolate caramel1 = new Caramel();
```

In the table below,

- The output produced by the statement in the left-hand column, should be written in the right-hand column
- If the statement produces more than one line of output, indicate the line breaks with slashes as in "a/b/c" to indicate three lines of output with "a" followed by "b" followed by "c".
- If the statement causes an error, fill in the right-hand column with either the phrase "compiler error" or "runtime error" to indicate when the error would be detected.-

	Statement	Output
1	choco1.method1();	
2	dairyMilk1.method1();	
3	dairyMilk2.method4();	
4	caramel1.method1();	

5	System.out.println(caramel1);	
6	System.out.println(caramel1.texture);	
7	((Chocolate)kit).method2();	
8	((SilkOreo)dairyMilk2).method3();	
9	((DairyMilk)kit).method2();	
10	((Chocolate)kit).method2();	
11	((Chocolate)dairyMilk2).method1();	
12	((Chocolate)obj1).method2();	
13	((Caramel)obj1).method2();	
14	((SilkOreo)obj2).method3();	
15	System.out.println(((Object)choco1).toString());	
16	System.out.println(((Chocolate)kit).texture);	

HOMEWORK

[SUBMISSION DEADLINE: 10 MAY, 11:59 PM]

Task 1

Write the **CSEStudent** and **CSE111Student** classes derived from **Student** class so that the following code generates the output below:

Parent Class	
<pre>public class Student{ public String msg = "I love BU"; public String shout(){ return msg; } }</pre>	
Driver Code	Output
<pre>public class StudentTester{ public static void printShout(Student s){ System.out.println("-----"); System.out.println(s.msg); System.out.println(s.shout()); } public static void main(String [] args){ Student s = new Student(); CSEStudent cs = new CSEStudent(); CSE111Student cs111 = new CSE111Student(); System.out.println(s.msg); System.out.println(cs.msg); System.out.println(cs111.msg); printShout(s); printShout(cs); printShout(cs111); } }</pre>	<pre>I love BU I want to transfer to CSE I love Java Programming ----- I love BU I love BU ----- I love BU I want to transfer to CSE ----- I love BU I love Java Programming</pre>

Task 2

Write the **PlatinumCard** and **SignatureCard** classes derived from **CreditCard** class so that the following code generates the output below.

Note: Platinum card users initially have 100 reward points and will get 2 reward points for spending 100 taka each. Signature card users initially have 200 reward points and will get 4 reward points for spending 100 taka each. Signature card users are allowed to bring upto 5 companions at lounges.

Parent Class	
<pre>public class CreditCard { public String cardHolder; public String accountNo; public int rewardPoints; public CreditCard(String cardHolder, String accountNo, int rewardPoints){ this.cardHolder = cardHolder; this.accountNo = accountNo; this.rewardPoints = rewardPoints; } public void cardDetails(){ System.out.println("Card Holder Name: " + cardHolder); System.out.println("Account Number: " + accountNo); System.out.println("Reward point gained: " + rewardPoints); } }</pre>	
Driver Code	Output
<pre>public class CardTester { public static void main(String[] args) { CreditCard card1 = new PlatinumCard("Ali", "345 127"); CreditCard card2 = new SignatureCard("Rahul", "514 123"); CreditCard card3 = new SignatureCard("Rohan", "147 965"); CreditCard [] cards = {card1, card2, card3}; for (int i = 0; i<cards.length; i++) { System.out.println("====="); if (cards[i] instanceof SignatureCard) { SignatureCard new_card = (SignatureCard) cards[i]; new_card.spendCash(500); } else if (cards[i] instanceof PlatinumCard) { PlatinumCard new_card = (PlatinumCard) cards[i]; new_card.spendCash(200); } System.out.println("====="); cards[i].cardDetails(); } } }</pre>	<pre>===== Previous Reward Points: 100 Reward points after spending 200 taka: 104 ===== Card Holder Name: Ali Account Number: 345 127 Reward point gained: 104 ===== Previous Reward Points: 200 Reward points after spending 500 taka: 220 ===== Card Holder Name: Rahul Account Number: 514 123 Reward point gained: 220 Possible Number of Companions for Lounge: 5 ===== Previous Reward Points: 200 Reward points after spending 500 taka: 220 ===== Card Holder Name: Rohan Account Number: 147 965 Reward point gained: 220 Possible Number of Companions for Lounge: 5</pre>

Task 3

1	public class Sue {
2	void method1() {
3	System.out.println("sue 1");
4	}
5	void method3() {
6	System.out.println("sue 3");
7	}
8	}
9	
10	public class Blue {
11	void method1() {
12	System.out.println("blue 1");
13	method3();
14	}
15	void method3() {
16	System.out.println("blue 3");
17	}
18	}
19	
20	public class Moo extends Blue {
21	void method2() {
22	super.method3();
23	System.out.println("moo 2");
24	this.method3();
25	}
26	void method3() {
27	System.out.println("moo 3");
28	}
29	}
30	
31	public class Crew extends Moo {
32	void method1() {
33	System.out.println("crew 1");
34	}
35	void method3() {
36	System.out.println("crew 3");
37	}
38	}

Assuming the following variables have been defined:

```
Moo var1 = new Crew();  
Blue var2 = new Moo();  
Object var3 = new Sue();  
Sue var4 = new Sue();
```

```
Blue var5 = new Crew();
Blue var6 = new Blue();
```

In the table below,

- The output produced by the statement in the left-hand column, should be written in the right-hand column
- If the statement produces more than one line of output, indicate the line breaks with slashes as in "a/b/c" to indicate three lines of output with "a" followed by "b" followed by "c".
- If the statement causes an error, fill in the right-hand column with either the phrase "compiler error" or "runtime error" to indicate when the error would be detected.

	Statement	Output
1	var1.method1();	
2	var2.method1();	
3	var3.method1();	
4	var4.method1();	
5	var5.method1();	
6	var6.method1();	
7	var1.method3();	
8	var2.method3();	
9	var3.method3();	
10	((Blue)var1).method1();	
11	((Crew)var1).method2();	
12	((Sue)var1).method3();	
13	((Blue)var3).method1();	
14	((Crew)var3).method1();	
15	((Sue)var3).method3();	
16	((Moo)var2).method2();	
17	((Crew)var3).method2();	
18	((Moo)var5).method2();	
19	((Moo)var6).method2();	
20	((Moo)var2).method1();	

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Write the Garage, Bike and Car class. **Car**, **Bike** are child classes of **Vehicle** class. But **Garage** is neither a parent nor a child class. The Garage class has **two arrays as instance variables** called *cars* and *bikes* that can store **Car and Bike objects**.

Hint: In this task you'll need to use the **instanceof** keyword and **downcasting**.

Parent Class

```
public class Vehicle {  
    private String brand;  
    private int year, wheels;  
  
    public Vehicle(String b, int y){  
        this.brand = b;  
        this.year = y;  
    }  
  
    public String getBrand(){  
        return this.brand;  
    }  
  
    public int getYear(){  
        return this.year;  
    }  
  
    public void setWheels( int w ){  
        this.wheels = w;  
    }  
  
    public int getWheels(){  
        return this.wheels;  
    }  
  
    public String toString(){  
        return "Brand: "+this.brand+", Year: "+this.year+", Wheels: "+this.wheels;  
    }  
}
```

DRIVER CODE	OUTPUT
<pre> Garage g = new Garage(2, 3); System.out.println("=====0====="); Vehicle vC1 = new Car("Ford", "Mustang", 2022, 2, 4, false); Vehicle vC2 = new Car("Tesla", "Model S", 2025, 4, 4, true); Vehicle vC3 = new Car("Reliant", "Robin", 1981, 2, 3, false); System.out.println("=====1====="); System.out.println(vC1); System.out.println("=====2====="); g.addVehicle(vC1); g.addVehicle(vC2); g.addVehicle(vC3); System.out.println(g.cars[1]); System.out.println("=====3====="); g.cars[0].startAutoPilot(); g.cars[1].startAutoPilot(); System.out.println("=====4====="); Vehicle vB1 = new Bike("Honda", "Gold Wing", 2022, 3, true); System.out.println(vB1); g.addVehicle(vB1); System.out.println("=====5====="); Vehicle vB2 = new Bike("Royal Enfield", "Classic 350", 2021, 2, false); g.addVehicle(vB2); System.out.println(g.bikes[1]); System.out.println("=====6====="); Vehicle vB3 = new Bike("Harley-Davidson", "Street 750", 2022, 2, false); g.addVehicle(vB3); Vehicle vB4 = new Bike("Yamaha", "MT-15", 2023, 2, false); g.addVehicle(vB4); System.out.println("=====7====="); g.bikes[0].doAWheelie(); g.bikes[1].doAWheelie(); </pre>	<pre> Welcome to the Garage! Car Capacity: 2 Bike Capacity: 3 =====0===== =====1===== Car Brand: Ford, Year: 2022, Wheels: 4, Model: Mustang, Doors: 2, AI: false =====2===== A Ford CAR has been added to the Garage A Tesla CAR has been added to the Garage Can't add more Cars! Capacity: 2 Car Brand: Tesla, Year: 2025, Wheels: 4, Model: Model S, Doors: 4, AI: true =====3===== Ford:Mustang has NO AutoPilot Tesla:Model S AutoPilot Started =====4===== Bike Brand: Honda, Year: 2022, Wheels: 3, Model: Gold Wing, SideCar: true A Honda BIKE has been added to the Garage =====5===== A Royal Enfield BIKE has been added to the Garage Bike Brand: Royal Enfield, Year: 2021, Wheels: 2, Model: Classic 350, SideCar: false =====6===== A Harley-Davidson BIKE has been added to the Garage Can't add more bikes! Capacity: 3 =====7===== Wheelie Failed. Honda:Gold Wing has SideCar Royal Enfield:Classic 350 is doing Wheelie!! </pre>

Task 2

Design the **SmartSecurityCamera** class derived from SmartDevice class to generate the following output.

Tester Code and Parent Class	Output
<pre>public class SmartHomeTester { public static void main(String[] args) { SmartSecurityCamera cam1 = new SmartSecurityCamera("Garden-Cam", 100, 64); cam1.powerOn(); System.out.println("====="); cam1.record(true); System.out.println("====="); cam1.powerOff(); System.out.println("====="); cam1.powerOn(); System.out.println("====="); cam1.record(); System.out.println("====="); cam1.formatCard("0000"); System.out.println("====="); cam1.formatCard("ADMIN123"); System.out.println("====="); SmartSecurityCamera cam2 = new SmartSecurityCamera("Indoor-Cam", 80, 1); cam2.powerOn(); System.out.println("====="); cam2.record(); System.out.println("====="); cam2.powerOff(); System.out.println("====="); cam2.powerOn(); System.out.println("====="); cam2.formatCard("ADMIN123"); System.out.println("====="); cam2 = new SmartSecurityCamera("Indoor-Cam", 2, 10); cam2.powerOn(); } } class SmartDevice { public String deviceName; private double batteryLevel; protected boolean isActive; SmartDevice(String name, double battery) { this.deviceName = name; this.batteryLevel = battery; this.isActive = false; } public void powerOn() { if (batteryLevel > 5) { isActive = true; batteryLevel -= 2; System.out.println(deviceName + " is now ONLINE."); } } }</pre>	<pre>Garden-Cam is now ONLINE. ===== [IR SENSORS ACTIVE] Recording standard footage. ===== Turning off Night Vision. Garden-Cam has shut down. ===== Garden-Cam is now ONLINE. ===== Recording standard footage. ===== REQUEST: Format SD Card initiated. ACCESS DENIED: Incorrect PIN. ===== REQUEST: Format SD Card initiated. Auth Success. Wiping data. SUCCESS: Storage restored to 64GB. ===== Indoor-Cam is now ONLINE. ===== Recording standard footage. ===== Indoor-Cam has shut down. ===== Error: Indoor-Cam storage full. Recording disabled. ===== REQUEST: Format SD Card initiated. ERROR: Device must be ON to format. ===== Power Low: Indoor-Cam cannot start.</pre>

```
    } else {  
        System.out.println("Power Low: " + deviceName + "  
cannot start.");  
    }  
}  
  
public void powerOff() {  
    this.isActive = false;  
    System.out.println(deviceName + " has shut down.");  
}  
  
public double getBattery() {  
    return batteryLevel;  
}  
}
```

Task 3

1	public class Foo {
2	String name = "foo";
3	public void call1() {
4	System.out.println("Foo 1");
5	}
6	public void call2() {
7	call1();
8	System.out.println("Foo 2");
9	}
10	}
11	
12	public class Bar extends Foo {
13	public void call2() {
14	System.out.println("Bar 2");
15	}
16	public void call3() {
17	System.out.println("Bar 3");
18	}
19	}
20	
21	public class Buzz extends Bar {
22	String name = "Buzz";
23	public void call1() {
24	System.out.println("Buzz 1");
25	}
26	public void call4() {
27	call3();
28	System.out.println("Buzz 4");
29	}
30	}
31	public class Bux extends Foo {
32	String name = "Bux";
33	public void call1() {
34	System.out.println("Bux 1");
35	}
36	public void call3() {
37	System.out.println("Bux 3");
38	}
39	}

Assuming the following variables have been defined:

```
Foo foo1 = new Foo();  
Bar bar1 = new Bar();
```

```

Bux bux1 = new Bux();
Foo foo2 = new Buzz();
Bar bar2 = new Buzz();
Object obj1 = new Foo();

```

In the table below,

- The output produced by the statement in the left-hand column, should be written in the right-hand column
- If the statement produces more than one line of output, indicate the line breaks with slashes as in "a/b/c" to indicate three lines of output with "a" followed by "b" followed by "c".
- If the statement causes an error, fill in the right-hand column with either the phrase "compiler error" or "runtime error" to indicate when the error would be detected.

	Statement	Output
1	bar1.call1();	
2	foo2.call1();	
3	foo2.call2();	
4	bar2.call3();	
5	System.out.println(bar1.name);	
6	System.out.println(bar2.name);	
7	System.out.println(((Buzz)bar2).name);	
8	((Buzz)bar1).call4();	
9	((Bar)foo1).call3();	
10	((Foo)bux1).call1();	
11	((Bux)foo1).call1();	
12	bux1.call1();	
13	bux1.call2();	
14	((Foo)foo2).call2();	
15	((Buzz)obj1).call3();	
16	((Buzz)obj1).call2();	
17	((Bux)foo2).call2();	
18	((Buzz)obj1).call1();	
19	System.out.println(foo2.name);	
20	System.out.println(((Bux)foo2).name);	

Task 4

Write the **UsedBattery** and the **PlasticBottle** classes derived from **Waste** class so that the following code generates the output below.

You also need to **complete the SorterBot class** for this code to work. You cannot create any additional methods in the **SorterBot** class.

Tester	Output
<pre>public class RecyclingPlant { public static void main(String[] args) { SorterBot robot = new SorterBot(); UsedBattery bat1 = new UsedBattery("Duracell-X", true); PlasticBottle bot1 = new PlasticBottle("Coke-Zero", 500); Waste unknown = new Waste("Mystery-Box"); System.out.println("#####"); robot.processItem(bat1); robot.processItem(bot1); robot.processItem(unknown); robot.processItem(new UsedBattery("Energizer", false)); } } //Parent Class class Waste { String id; Waste(String id) { this.id = id; } } //Disjoint Class class SorterBot { public void processItem(Waste item) { //WRITE YOUR CODE HERE System.out.println("-----"); } }</pre>	<pre>##### Scanning Item Duracell-X [Type: BATTERY] Duracell-X is leaking. Sealing in concrete container. ----- Scanning Item Coke-Zero [Type: PLASTIC] Coke-Zero compressed from 500ml to flat disk. ----- Scanning Item Mystery-Box [Type: UNKNOWN] Item sent to generic incinerator. ----- Scanning Item Energizer [Type: BATTERY] Energizer stored in dry cell. -----</pre>

Task 5

Your task is to design the **StudyRoom** class with appropriate variables and methods such that the following tester code produces the expected output. Note:

- Assume that each **StudyRoom** can add two books.
- You cannot use any arrays in the **StudyRoom** class.
- You should use the given **Library** and **SilentStudyRoom** classes' variables and methods as needed.
- You cannot modify the given **Library** and **SilentStudyRoom** classes.

Tester Code	Expected Output
<pre>public class TestLibrary{ public static void main(String[] args) { Library library = new Library("The Mind Maze"); library.showRoomInfo(); System.out.println("===== 1 ====="); StudyRoom room9 = new StudyRoom("Study Hub"); SilentStudyRoom room9A = new SilentStudyRoom("Focus Room"); room9.addBook("Data Structures"); System.out.println("===== 2 ====="); room9.addBook("Operating Systems"); room9.showRoomInfo(); System.out.println("===== 3 ====="); System.out.println("Total books: " +StudyRoom.totalBooks); System.out.println("===== Add Book ====="); room9A.addBook(library, room9); System.out.println("===== 4 ====="); room9A.showRoomInfo(); System.out.println("Total books: " +StudyRoom.totalBooks); System.out.println("===== Add Book Again ====="); room9A.addBook(library, room9); System.out.println("===== 5 ====="); room9A.showRoomInfo(); System.out.println("Total books: " + StudyRoom.totalBooks); } }</pre>	<pre>Library Name: The Mind Maze ===== 1 ===== ===== 2 ===== Study Hub Details: Book 1: Data Structures Book 2: Operating Systems ===== 3 ===== Total books: 2 ===== Add Book ===== Library Name: The Mind Maze Adding book: Data Structures Book 1 Removed from Study Hub Book added successfully! ===== 4 ===== Focus Room Details: Book 1: Data Structures No book added Total books: 1 ===== Add Book Again ===== Library Name: The Mind Maze Adding book: Operating Systems Book 2 Removed from Study Hub Book added successfully! ===== 5 ===== Focus Room Details: Book 1: Data Structures Book 2: Operating Systems Total books: 0</pre>
<pre>// Grand Parent Class class Library{ public String name; public Library(String name) { this.name = name; } public void showRoomInfo() { System.out.println("Library Name: " + name); } }</pre>	
<pre>// Parent Class</pre>	

```
class StudyRoom extends Library{  
    // Write Your Code Here  
}
```

```
// Child Class  
class SilentStudyRoom extends StudyRoom{  
  
    public SilentStudyRoom(String name) {  
        super(name);  
    }  
  
    public void addBook(Library lib, StudyRoom room) {  
        lib.showRoomInfo();  
        if (room.getBook1() != null) {  
            System.out.println("Adding book: " + room.getBook1());  
            this.setBook1(room.getBook1());  
            room.removeBook(1);  
            System.out.println("Book added successfully!");  
        }  
        else if (room.getBook2() != null) {  
            System.out.println("Adding book: " + room.getBook2());  
            this.setBook2(room.getBook2());  
            room.removeBook(2);  
            System.out.println("Book added successfully!");  
        }  
        else {  
            System.out.println("No books available in "+room.name);  
        }  
    }  
}
```

Task 6

Your task is to design the **UpsideDown** class with appropriate variables and methods such that the following tester code produces the expected output. Note:

- Assume that each gate of **UpsideDown** can connect with two bridges.
- You cannot use any arrays in the **UpsideDown** class.
- You should use the given **Hawkins** and **DarkDimension** classes' variables and methods as needed.
- You cannot modify the given **Hawkins** and **DarkDimension** classes.

Tester Code	Expected Output
<pre>public class HawkinsLabTester { public static void main(String[] args) { Hawkins place1 = new Hawkins("Hawkins Lab"); Hawkins place2 = new Hawkins("Palace Arcade"); UpsideDown gate1 = new UpsideDown("The Nina Project"); UpsideDown gate2 = new UpsideDown("Brimborn Steel Works"); DarkDimension world = new DarkDimension("The Dark World"); gate1.open(); System.out.println("Total bridges: " + UpsideDown.totalBridges); System.out.println("===== [1] ====="); gate1.connect(place1); gate1.connect(place2); Hawkins place3 = new Hawkins("Starcourt Mall"); gate2.connect(place3); gate1.details(); gate2.details(); System.out.println("===== [2] ====="); world.runExperiment(gate1); world.runExperiment(gate2); System.out.println("===== [3] ====="); System.out.println("Total bridges: " + UpsideDown.totalBridges); System.out.println("===== [4] ====="); Hawkins place4 = new Hawkins("Byers new house"); gate1.connect(place4); gate1.disconnect(2); gate2.disconnect(3); System.out.println("===== [5] ====="); gate1.details(); } }</pre>	<pre>Bridge from The Nina Project is Open Total bridges: 0 ===== [1] ===== The Nina Project Details: Bridge 1: Hawkins Lab Bridge 2: Palace Arcade Brimborn Steel Works Details: Bridge 1: Starcourt Mall ===== [2] ===== Bridge present at The Nina Project Activating the door of Hawkins Lab Experiment executed successfully! No Bridge present at Brimborn Steel Works Cannot run experiment. ===== [3] ===== Total bridges: 3 ===== [4] ===== No further bridges with The Nina Project Invalid bridge number! ===== [5] ===== The Nina Project Details: Bridge 1: Hawkins Lab</pre>
<pre>// Grand Parent Class class Hawkins{ public String name; public boolean status=false; public Hawkins(String name) { this.name = name; } public boolean checkBridge(Hawkins h) { if (h.status==true) { System.out.println("Bridge present at " + h.name); return true; } else { System.out.println("No Bridge present at " + h.name); return false; } } }</pre>	

```

    }
}

public void open() {
    if (status==false){
        status = true;
        System.out.println("Bridge from "+name+" is Open");
    }
}
}

```

```

// Parent Class
class UpsideDown extends Hawkins{
    // Write Your Code Here
}

```

```

// Child Class
class DarkDimension extends UpsideDown {
    public DarkDimension(String name) {
        super(name);
    }

    public void runExperiment(UpsideDown portal) {
        if (!this.checkBridge(portal)) {
            System.out.println("Cannot run experiment.");
        }
        else {
            if (portal.getBridge1() != null) {
                portal.activate(portal.getBridge1());
                System.out.println("Experiment executed successfully!");
            } else if (portal.getBridge2() != null) {
                portal.activate(portal.getBridge2());
                System.out.println("Experiment executed successfully!");
            } else {
                System.out.println("No experiment found!");
            }
        }
    }
}
}

```